

Coding Lab

Working with biological data on the Linux command line



Bartosz Czech, PhD, Eng.

Department of Genetics

Wrocław University of Environmental and Life Sciences

Contents

Preface	1
About this course	1
What you will learn	1
How to use this book	1
The example data	2
Setting up your environment	2
1 Elements of the Linux command line	3
1.1 The role of the terminal	3
1.2 The structure of a command and how to get help	4
1.3 Navigating the filesystem and basic operations	5
1.4 File permissions	8
1.5 Links	9
1.6 Common mistakes	9
1.7 Command summary	10
1.8 Exercises	11
2 An overview of command-line tools	12
2.1 How the shell expands a command	12
2.2 Streams and redirection	14
2.3 Pipelines	14
2.4 Text filters	15
2.5 Searching with grep and regular expressions	17
2.6 Compression and archives	18
2.7 Common mistakes	19
2.8 Command summary	19
2.9 Exercises	21
3 The VIM text editor	22
3.1 The modal model	22
3.2 The VIM screen	23
3.3 Surviving: open, insert, save, quit	23
3.4 Moving around in Normal mode	24

3.5	Editing text	24
3.6	Searching and substitution	25
3.7	Working with files and a first configuration	25
3.8	nano: the simple alternative	25
3.9	Common mistakes	26
3.10	Command summary	26
3.11	Exercises	27
4	Biological data: databases and formats	28
4.1	Where data comes from: databases and accessions	28
4.2	Downloading and verifying files	29
4.3	FASTA: sequences	29
4.4	FASTQ: reads with quality	30
4.5	Working with compressed files	31
4.6	SAM/BAM: alignments	31
4.7	VCF: variants	32
4.8	GTF/GFF3: annotation	33
4.9	BED and coordinate systems	34
4.10	Common mistakes	35
4.11	Command summary	35
4.12	Exercises	37
5	The AWK programming language	38
5.1	The data model: records and fields	38
5.2	The pattern-action model	39
5.3	Running AWK and setting the field separator	40
5.4	Patterns: choosing the lines	41
5.5	Computing: arithmetic, sums and <code>printf</code>	41
5.6	Associative arrays: counting and grouping	42
5.7	Putting it together: BEGIN, END and a real statistic	42
5.8	Recipes for biological data	43
5.9	A companion tool: <code>sed</code>	44
5.10	Common mistakes	44
5.11	Command summary	45
5.12	Materials	45
5.13	Exercises	46

6	Bash as a scripting language	47
6.1	Anatomy of a script	47
6.2	Variables and quoting	48
6.3	Positional arguments	48
6.4	Transforming file names	49
6.5	Exit status and combining commands	49
6.6	Conditionals	50
6.7	Loops	51
6.8	Arithmetic	53
6.9	Functions	53
6.10	Arrays	54
6.11	Putting it together	55
6.12	Common mistakes	55
6.13	Command summary	56
6.14	Materials	57
6.15	Exercises	58
7	Implementing software in the Linux environment	59
7.1	From a script to a tool: options with getopt	59
7.2	Validating input and failing loudly	60
7.3	Running safely: set -euo pipefail	60
7.4	Cleaning up: trap and mktemp	62
7.5	Finding mistakes: bash -n, set -x and shellcheck	63
7.6	Combining programs: process substitution	63
7.7	Joining tables by a shared key: join	64
7.8	Doing work in parallel: xargs and GNU parallel	65
7.9	Version control with Git	65
7.10	Reproducible environments: Conda and Docker	66
7.11	Putting it together: a complete tool	67
7.12	Common mistakes	69
7.13	Command summary	69
7.14	Materials	70
7.15	Exercises	71

8	Writing custom scripts for biological data processing	72
8.1	The shape of an analysis	72
8.2	Branch A: gene names from the annotation	72
8.3	Branch B: fold change from the counts	73
8.4	Merging the branches: join, classify, sort	73
8.5	From commands to a tool	74
8.6	Making it reproducible	76
8.7	Common mistakes	77
8.8	Materials	77
8.9	Exercises	78

Preface

About this course

Coding Lab teaches you to work on the **Linux command line**, the environment in which most biological data is processed today. You will learn to move around a Unix system, combine small command-line tools into larger analyses, edit files in a terminal editor, read the standard file formats used in genomics, write programs in **AWK**, turn a sequence of commands into a reusable **Bash script**, and package that script so that it runs reliably for someone else.

The book does not assume that you have programmed before, and it does not assume a background in biology. Readers come to this material from different degrees (biology, computer science, biotechnology, chemistry), and it is written so that all of them can follow it. Where a command operates on biological data, a short note in plain language explains what the data is and why the question matters. You do not need to memorise biology to work through the book; you need to learn to manipulate files and text reproducibly, which is a transferable skill.

What you will learn

The chapters follow the course syllabus:

1. **Elements of the Linux command line:** the shell, command structure, the filesystem, and file permissions.
2. **Overview of command-line tools:** pipelines, text filters, searching with **grep**, and working with compressed files.
3. **The VIM text editor:** editing files without leaving the terminal.
4. **Biological database formats and resources:** FASTA, FASTQ, SAM/BAM, VCF, GTF/GFF and BED, and the public databases they come from.
5. **The AWK programming language:** column-oriented processing of tabular biological data.
6. **Bash as a scripting language:** variables, conditionals, loops, and functions, assembled into scripts.
7. **Implementing software in the Linux environment:** writing scripts that are robust, documented, and reproducible for others.
8. **Writing custom scripts for biological data processing:** a capstone that combines everything into a small analysis pipeline.

How to use this book

Read each chapter at a terminal and type the commands as you go, reading alone is not enough to learn a command line. Commands and their output are shown in monospaced blocks:

```
echo "Hello, Coding Lab"
```

Hello, Coding Lab

Every chapter ends with a short set of **exercises** that use the course data. Work through them before moving on; later chapters build on earlier ones.

A note on the biology. Short boxes like this one give the minimum context needed to understand an example (what a gene, a read, or a variant is) and why someone would count or filter it. They are background, not material to be memorised.

The example data

All examples use a single data package, `coding_lab_data.zip`, available on the course page. Download and unpack it once:

```
unzip coding_lab_data.zip
cd coding_lab_data
```

The data is **real** (gene annotations, sequences, sequencing reads and variants drawn from public genomic resources), reduced to a size that is convenient to work with. The numbers shown in this book were computed on exactly these files, so if you run the same commands you will obtain the same results.

Setting up your environment

You need a working Unix shell (**bash**) with the standard GNU command-line tools.

- **Linux:** everything is already available. Open a terminal and begin.
- **macOS:** a terminal is built in. Some examples use GNU versions of tools (for instance **grep**, **sed**, **awk**); the simplest way to obtain them, together with the heavier bioinformatics tools used in Chapter 4, is the **conda** package manager (see Chapter 7).
- **Windows:** install the **Windows Subsystem for Linux** (WSL2) and a Linux distribution such as Ubuntu from the Microsoft Store. All work in this course is done inside that Linux environment, not in PowerShell.

Instructions for installing the specialised tools (such as **samtools** and **bedtools**) appear in the chapters that first use them.

Chapter 1

Elements of the Linux command line

This chapter introduces the environment in which the rest of the course takes place: the **command line**. You will learn what the terminal and the shell are, how a command is built and how to ask the system for help, how to move around the filesystem, and how to read and change the permissions that control who may use a file. These are the foundations on which every later chapter rests.

Why the command line for biological data? A sequencing run produces files that are far too large to open in a spreadsheet, and an analysis usually means running the *same* step on tens or hundreds of such files. The command line is built for exactly this: a short, precise instruction can be applied to one file or to a thousand, and written down so that it can be repeated and checked.

1.1 The role of the terminal

When you work on a Linux system without a graphical interface, you type instructions as text and the computer answers with text. Three words for the parts of this arrangement are often used loosely, but it is worth keeping them apart.

- The **terminal** (or *terminal emulator*) is the window in which text appears. It is only a display: it shows what you type and what the system prints back.
- The **shell** is the program that reads your line of text, interprets it, runs the requested command, and returns the result. It is the interpreter sitting behind the terminal.
- **Bash** (*Bourne Again Shell*) is the most common shell on Linux, and the one used throughout this course. Other shells exist (**zsh**, **fish**, **dash**) and differ in their convenience features, but the core ideas are shared.

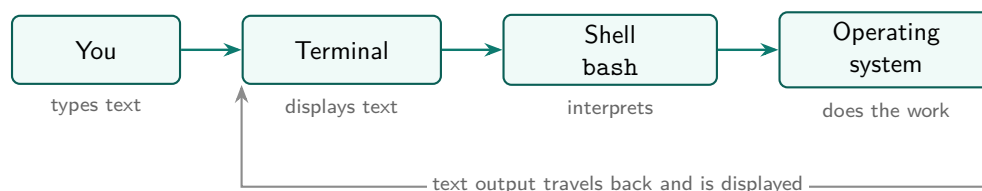


Figure 1.1: The round trip of a command. You type text into the **terminal**; the **shell** interprets it and asks the **operating system** to carry it out; any output travels back along the same path.

You can ask which shell you are currently running:

```
echo $SHELL
```

```
/bin/bash
```

The `$SHELL` here is a **variable**, a name that stands for a value the system keeps for you (variables are covered in detail in Chapter 6). The `echo` command simply prints the value. If the answer is `/bin/zsh` rather than `/bin/bash`, most of this course still applies unchanged, but the examples assume Bash.

The text that appears before your cursor is the **prompt**. Its exact form varies between systems; in this book the prompt is shown as `$` and is *not* part of what you type. A line beginning with `$` is a command to enter; the lines that follow it, without a `$`, are the output the system produced.

1.2 The structure of a command and how to get help

Almost every command you type follows the same pattern:

```
command [options] [arguments]
```

- the **command** is the name of the program to run (for example `ls`);
- **options** (also called *flags*) change how it behaves (for example `-l`);
- **arguments** are the things it acts on (for example a file name).

Consider:

```
ls -l -h transcripts.fa
```

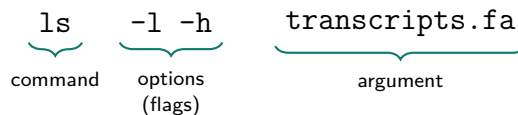


Figure 1.2: *The anatomy of a command. The first word names the program to run; options change how it behaves; arguments are the things it acts on.*

Here `ls` is the command, `-l` and `-h` are options, and `transcripts.fa` is the argument. Options come in two styles:

- **short options** are a single letter after one dash: `-l`, `-h`. Several short options can usually be combined, so `-l -h` may be written `-lh`.
- **long options** are a whole word after two dashes: `--human-readable`. They are more verbose but easier to read and to remember.

Some options themselves take a value, written either with a space or an equals sign, `--width 80` or `--width=80`. Finally, two dashes on their own, `--`, mark the end of the options; anything after them is treated as an argument even if it begins with a dash. This matters when, for example, a file is accidentally named `-report.txt`.

1.2.1 Asking the system for help

You will not remember every option, and you are not expected to. Linux documents itself, and learning to read that documentation is more valuable than memorising commands.

The **manual pages** describe a command in full:

```
man ls
```

The manual opens in a pager (press **q** to quit, the arrow keys or *space* to scroll, and **/word** to search for *word*). Many commands also accept a **--help** option that prints a shorter summary directly:

```
ls --help
```

If you do not know the *name* of the command you need, search the manuals by keyword with **-k** (apropos):

```
man -k compress
```

```
gzip (1)          - compress or expand files
gunzip (1)        - compress or expand files
zcat (1)          - compress or expand files
...
```

Two further commands answer “what is this name?”:

- **type** *ls* tells you whether a name is a program, a shell built-in, or an alias;
- **which** *samtools* prints the full path to the program that would run.

1.2.2 Working efficiently

Two habits save a great deal of typing:

- **Tab completion.** Press *Tab* and the shell completes the name of a command or file as far as it is unambiguous; press it twice to list the possibilities. Begin to type **transc** and *Tab*, and the shell completes it to **transcripts.fa**. This also prevents spelling mistakes in long file names.
- **Searching your history.** Press *Ctrl-R* and begin typing part of an earlier command; the shell searches backwards through what you have run. Press *Enter* to run the match, or the arrow keys to edit it first.

1.3 Navigating the filesystem and basic operations

The Linux filesystem is a single tree. At its base is the **root** directory, written **/**, and every file and directory hangs somewhere beneath it. At any moment your shell sits in one directory, the **working directory**.

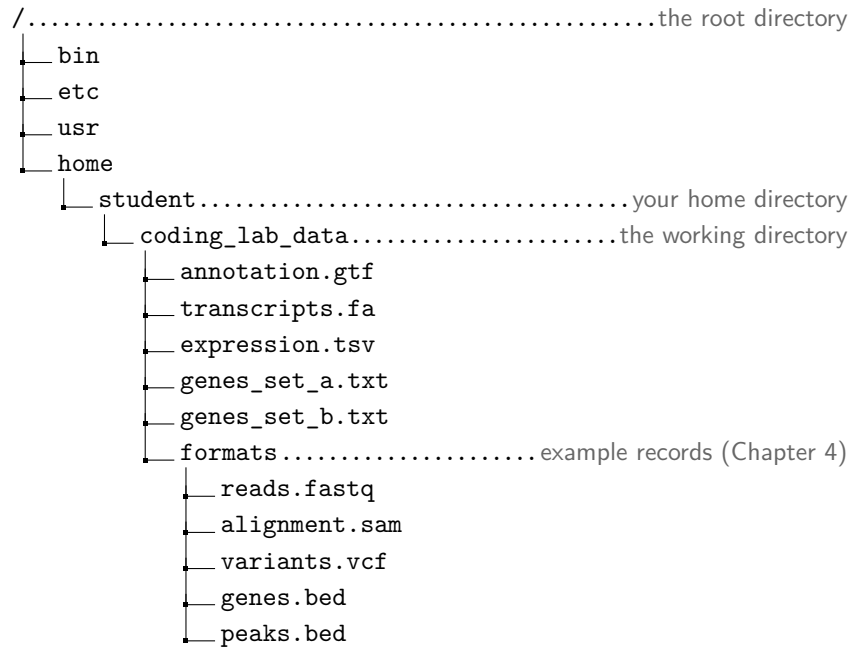


Figure 1.3: The filesystem is a single tree. Every file and directory hangs somewhere below the root `/`; at any moment the shell is “in” one directory, the working directory.

```
/home/student/coding_lab_data
```

`pwd` (“print working directory”) shows where you are. To see what is there, use `ls`:

```
ls
```

```

annotation.gtf  expression.tsv  formats  genes_set_a.txt  genes_set_b.txt
↪ transcripts.fa

```

`ls` becomes far more informative with options. The most useful combination is `-l` (one entry per line, with details), `-a` (include hidden entries, whose names begin with a dot), and `-h` (sizes in human-readable units such as K and M):

```
ls -lah
```

```

drwxr-xr-x  3 student  staff   192B Oct  6 10:21 .
drwxr-xr-x  8 student  staff  256B Oct  6 10:21 ..
-rw-r--r--  1 student  staff   10M Oct  6 10:21 annotation.gtf
-rw-r--r--  1 student  staff  592B Oct  6 10:21 expression.tsv
drwxr-xr-x  7 student  staff  224B Oct  6 10:21 formats
-rw-r--r--  1 student  staff  449B Oct  6 10:21 genes_set_a.txt
-rw-r--r--  1 student  staff  463B Oct  6 10:21 genes_set_b.txt
-rw-r--r--  1 student  staff   11M Oct  6 10:21 transcripts.fa

```

Each long-listing line packs several fields together. Reading the first line:

```

-rw-r--r--  1  student  staff    11M   Oct  6 10:21  transcripts.fa
\-----/  |  \-----/  \---/  \---/  \-----/  \-----/
permissions links  owner    group   size    modified    name

```

The permission field is explained in the next section.

1.3.1 Moving around

You change directory with `cd`:

```

cd coding_lab_data
cd ..
cd ~
cd

```

A few shorthand names appear constantly and are worth learning at once:

Symbol	Meaning
.	the current directory
..	the parent directory
~	your home directory
/	the root of the filesystem
-	the previous directory (<code>cd -</code>)

Paths come in two kinds. An **absolute path** starts from the root and is valid from anywhere: `/home/student/coding_lab_data/transcripts.fa`. A **relative path** starts from the working directory: if you are in `coding_lab_data`, then `transcripts.fa` and `../coding_lab_data/transcripts.fa` both refer to the same file. Relative paths are shorter; absolute paths are unambiguous. Both are used constantly, and knowing which you are writing avoids a large share of beginner errors.

1.3.2 Creating, copying, moving and removing

Command	Action
<code>mkdir results</code>	create a directory
<code>mkdir -p a/b/c</code>	create a directory and any missing parents
<code>touch notes.txt</code>	create an empty file (or update its timestamp)
<code>cp a.txt b.txt</code>	copy a file
<code>cp -r dir1 dir2</code>	copy a directory and its contents (<code>-r</code> , recursive)
<code>mv a.txt data/</code>	move a file into a directory
<code>mv old.txt new.txt</code>	rename a file
<code>rm file.txt</code>	remove a file
<code>rm -r dir</code>	remove a directory and its contents

rm does not ask twice. There is no recycle bin on the command line: a removed file is gone. Pause before `rm -r`, and consider `rm -i`, which asks for confirmation on each file.

1.3.3 Looking at file contents

You will often want to inspect a file without opening an editor:

Command	Use
<code>cat file</code>	print the whole file
<code>less file</code>	view a long file page by page (<code>q</code> to quit, <code>/</code> search)
<code>head file</code>	first 10 lines (<code>head -n 3</code> for the first three)
<code>tail file</code>	last 10 lines (<code>tail -n 20</code> for the last twenty)
<code>wc -l file</code>	count the lines in a file

For the large text files common in genomics, `less`, `head` and `tail` are the right tools, printing a multi-gigabyte file to the screen with `cat` is rarely what you want.

1.4 File permissions

Linux is a multi-user system, and every file records who is allowed to do what with it. This is the permission field you saw in `ls -l`:

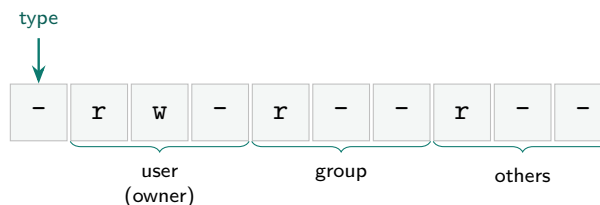


Figure 1.4: The ten-character permission field. The first character gives the file type (`-` a regular file, `d` a directory, `l` a symbolic link); the remaining nine are three triplets of read/write/execute permissions, for the owner, the group, and everyone else.

Within each of the three groups, three letters stand for three kinds of access:

- **r**, read (view the contents);
- **w**, write (change the contents);
- **x**, execute (run the file as a program, or, for a directory, enter it).

So `-rw-r--r--` means a regular file that the owner may read and write, while the group and everyone else may only read it.

1.4.1 Permissions as numbers

The same permissions are often written as three digits, because each of `r`, `w` and `x` has a numeric value:

Permission	Value
read (r)	4
write (w)	2
execute (x)	1

Adding the values for one group of users gives a single digit; three digits describe the whole file. For example 755 means 7 for the owner ($4 + 2 + 1 = \text{read, write, execute}$) and 5 for the group and others ($4 + 1 = \text{read and execute}$). Written out, 755 is **rwxr-xr-x**.

1.4.2 Changing permissions

chmod (“change mode”) sets permissions, in either notation:

```
chmod +x analyse.sh      # make a script executable (for everyone)
chmod 755 analyse.sh     # owner: all; group and others: read + execute
chmod u+w notes.txt      # add write permission for the owner (user)
chmod go-w shared.txt    # remove write for group and others
```

The symbolic form combines **who** (u user, g group, o others, a all), an **operation** (+ add, - remove, = set exactly), and the **permissions** (r, w, x). You will use **chmod +x** whenever you write your own script and want to run it, which happens from Chapter 6 onward.

1.5 Links

A **symbolic link** (symlink) is a small file that points at another file, somewhat like a shortcut. It is useful when the same large data file is needed in several places but should not be copied repeatedly:

```
ln -s /data/reference/genome.fa genome.fa
```

This creates **genome.fa** in the current directory, pointing at the real file under **/data/reference**. In a long listing a symlink is marked with **l** and an arrow showing its target:

```
lrwxr-xr-x  1 student staff  28 Oct 6 10:30 genome.fa ->
↪ /data/reference/genome.fa
```

1.6 Common mistakes

- **Spaces in names.** `cd My Data` is read as two arguments. Quote the name (`cd "My Data"`) or escape the space (`cd My\ Data`); better still, avoid spaces in file names altogether.
- **Confusing . and ..** `.` is here, `..` is one level up.
- **Forgetting -r for directories:** `cp` and `rm` need `-r` to act on a whole directory.
- **Case sensitivity.** `Genome.fa` and `genome.fa` are different files on Linux.
- **Assuming rm is reversible.** It is not; check the name before you press Enter.

1.7 Command summary

Task	Command
Which shell am I in	<code>echo \$SHELL</code>
Read the manual / short help	<code>man cmd · cmd --help</code>
Search manuals by keyword	<code>man -k keyword</code>
Where am I	<code>pwd</code>
List files (detailed)	<code>ls -lah</code>
Change directory	<code>cd path · cd .. · cd ~</code>
Make a directory	<code>mkdir -p path</code>
Copy / move / remove	<code>cp · mv · rm (-r for dirs)</code>
View a file	<code>less · head · tail · cat</code>
Count lines	<code>wc -l file</code>
Make a file executable	<code>chmod +x file</code>
Create a symbolic link	<code>ln -s target linkname</code>

1.8 Exercises

The exercises use the course data package (`coding_lab_data.zip`). Unpack it and change into the resulting directory before you begin.

1.1 Print your current working directory, then list its contents in long form with human-readable sizes and hidden entries shown. Which file is the largest?

1.2 Create a directory called `results`, and inside it an empty file called `log.txt`, using as few commands as you can. Confirm that both were created.

1.3 Using the manual, find out what the `-1` (the digit one) option to `ls` does, and what the `-S` option does. Then list the data directory sorted by size.

1.4 Copy `transcripts.fa` to `results/transcripts_backup.fa`. Then rename it to `results/reference.fa`. List `results` to confirm.

1.5 Look at the permission string of `transcripts.fa` in a long listing. Write it out in the three-digit numeric form, then state in words who may read and who may write the file.

1.6 Display only the first three lines of `annotation.gtf`, then only the last two. Finally, count how many lines the whole file has.

Chapter 2

An overview of command-line tools

Chapter 1 introduced the shell and the filesystem. This chapter is about *getting work done* with them. You will learn how the shell rewrites a command before it runs, how data flows through a program on three separate streams, how to join small programs into a **pipeline**, and how a handful of text filters (**cut**, **sort**, **uniq**, **grep** and a few others) combine to answer real questions about biological data. The chapter closes with the compression and archive tools you need because sequence files are large.

Why small tools joined together? Most biological data arrives as plain text in rows and columns: an annotation, a table of counts, a list of variants. The Unix tradition answers questions about such files not with one large program but with several small ones, each doing a single job well, piped one into the next. The same four-step pipeline that counts feature types in a yeast annotation will count read groups in an alignment or chromosomes in a variant file, unchanged.

2.1 How the shell expands a command

Before a program runs, the shell **expands** the line you typed: it replaces patterns with matching file names, `{...}` with a list of strings, and `$name` with a stored value. The program only ever sees the result of that expansion. A useful trick for seeing what the shell will do, without running the real command, is to put **echo** in front of it.

2.1.1 Globbing: matching file names

The characters `*`, `?` and `[...]` are replaced by the shell with file names in the current directory that match:

- `*`, any run of characters, including none: `*.fa` is every file ending in `.fa`;
- `?`, exactly one character: `chr?.bed` matches `chr1.bed` but not `chr12.bed`;
- `[...]`, one character from a set: `set_[ab].txt` matches `set_a.txt` and `set_b.txt`.

```
echo *.txt           # preview what the pattern expands to, before using it
ls genes_set_?.txt   # genes_set_a.txt genes_set_b.txt
```

This is **not** the regular-expression language used by **grep**. Globbing matches *file names*, and its characters mean different things (a glob `*` means “any text”; a regex `*` means “repeat the previous item”). If no file matches, the shell usually leaves the pattern untouched, so you see a literal `*.bam` instead of a list, a common surprise.

2.1.2 Brace expansion: generating strings

Braces expand into a list *before* file matching happens, and the files need not exist:

- {a,b,c}, an explicit list: `sample_{A,B,C}.txt` → three names;
- {1..5}, a range: {1..5} → 1 2 3 4 5;
- leading zeros are kept: {01..03} → 01 02 03.

```
echo sample_{A,B,C}      # sample_A sample_B sample_C
echo run_{1..3}.log     # run_1.log run_2.log run_3.log
```

Globbering *matches* existing files; brace expansion *creates* strings. The two are often combined, for instance to make a set of output directories in one command.

2.1.3 Command substitution and arithmetic

- `$(...)` is replaced by the output of the command inside it:

```
echo "Sequences in the file: $(grep -c '^>' transcripts.fa)"
```

The shell runs `grep -c ...` first and substitutes its result into the sentence.

- `$(( ... ))` evaluates simple integer arithmetic: `echo $(( 3 * 7 ))` prints 21.

2.1.4 Quoting: single, double, none

Quoting decides whether the shell expands what is inside:

- "...", variables and `$(...)` **are** expanded, but the text stays a single argument;
- '...', the content is **literal**; nothing is expanded;
- no quotes, expanded, and additionally split into words at spaces (risky).

```
name="Ada Lovelace"
echo "$name"      # Ada Lovelace    (one argument)
echo '$name'     # $name           (literal, no expansion)
echo $name       # Ada Lovelace    (but as TWO separate arguments!)
```

Always quote a variable as `"$var"`. Without quotes, a space in the value splits it into several arguments.

A common and costly mistake. With `file="final report.txt"`, the command `rm $file` deletes `final` and `report.txt` separately. Written correctly, `rm "$file"` removes the one intended file.

2.2 Streams and redirection

This is one of the central ideas of the whole course. Every running program is given three **streams**, channels through which data flows:

- **stdin** (stream 0), input, where the program reads data from;
- **stdout** (1), standard output, where it writes its result;
- **stderr** (2), a separate channel for error messages and warnings.

Keeping the result (stdout) apart from the errors (stderr) is deliberate: it lets you save the result to a file while still seeing the errors on screen, or the reverse.

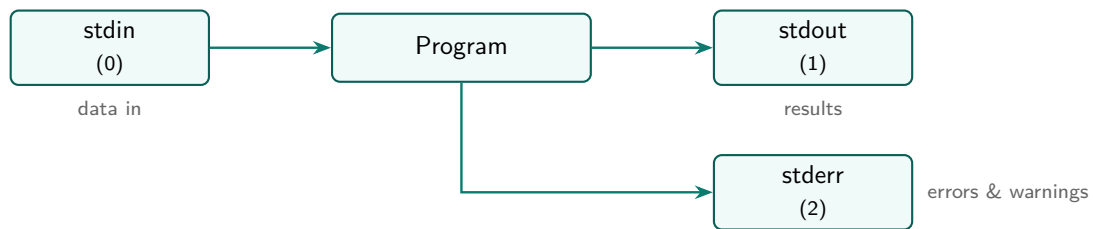


Figure 2.1: The three standard streams. A program reads from *stdin*, writes its results to *stdout*, and sends error messages to a separate channel, *stderr*, so that results and errors can be routed independently.

2.2.1 Redirection

Arrows send a stream to a file instead of the screen:

```

cmd > file           # stdout to a file (OVERWRITES existing contents)
cmd >> file          # stdout to a file (APPENDS to the end)
cmd 2> errors.log    # stderr to a file (the 2 sits tight against >)
cmd > out 2>&1        # stdout to a file, and stderr "to wherever stdout goes"
cmd 2>/dev/null      # discard errors into the system's "bin", /dev/null
  
```

Two traps. `>` erases the previous contents of the file without warning (use `>>` to add to it); and `cmd > file` captures only stdout, errors still appear on screen until stream 2 is redirected separately.

A short demonstration of the split, running `ls` on a file that exists and one that does not, at the same time:

```
ls transcripts.fa missing.txt > found.txt 2> errors.txt
```

The name of the file that exists goes to `found.txt` (stdout); the message `ls: missing.txt: No such file or directory` goes to `errors.txt` (stderr).

2.3 Pipelines

A **pipe** connects the stdout of one program directly to the stdin of the next, with no intermediate file. This is the foundation of the Unix philosophy: rather than one large tool, combine several small ones, each doing a single thing well. Data flows left to right through the stages.



Figure 2.2: A pipeline. Each stage reads what the previous one produced: `zcat` decompresses, `grep` keeps only the header lines (those starting with `>`), and `wc -l` counts them. The result is the number of sequences in a compressed FASTA file.

```
zcat transcripts.fa.gz | grep "^>" | wc -l
```

Each element processes only what the previous one handed it. When building a long pipeline, add one stage at a time and check the result after each (for example by previewing the start with `| head`), rather than writing the whole thing at once.

`tee` branches a stream: it writes the data to a file **and** passes it on, to the screen or the next stage, ... `| tee saved.txt`.

2.4 Text filters

A *filter* reads text from stdin, transforms it, and writes to stdout, the ideal pipeline component. Each of the following tools does one job; their power comes from combining them.

Tool	Use
<code>cut</code>	select columns
<code>sort</code>	order rows (<code>-n</code> numeric, <code>-k2</code> by the 2nd column)
<code>uniq -c</code>	count adjacent duplicates (always after <code>sort</code>)
<code>tr</code>	replace or delete characters
<code>paste/comm</code>	join files side by side / compare sorted lists
<code>column -t</code>	align columns into a readable table
<code>wc -l</code>	count lines

2.4.1 cut: selecting columns

By default `cut` splits on tabs; `-f` names the columns and `-d` changes the separator:

```
cut -f1,3 file.tsv      # columns 1 and 3
cut -f2- file.tsv      # from column 2 to the end
cut -d',' -f1 file.csv # first column of a CSV (separator: comma)
```

On the course expression table, selecting the gene name and the first control column:

```
cut -f1,2 expression.tsv | head -4
```

```
gene    control_1
YDL246C 142
YBR118W 2011
YAL003W 1567
```

2.4.2 sort: ordering rows

By default `sort` orders alphabetically, which puts 10 before 2. The most important options:

```
sort file           # alphabetical
sort -n file        # numeric (2 before 10)
sort -rn file       # numeric, descending
sort -k2,2n file    # numeric, by the 2nd column
sort -u file        # sort and drop duplicates
```

2.4.3 uniq: counting and removing repeats

`uniq` acts only on **adjacent** lines, so it is almost always preceded by `sort`:

```
sort file | uniq      # remove repeats
sort file | uniq -c    # count how many times each line occurred
sort file | uniq -d    # show only the lines that repeat
```

2.4.4 The canonical “count and rank” pattern

Cut out the column of interest, sort it, count identical values, and order the counts in descending order. For example, to count the **feature types** in a GTF annotation (column 3: `gene`, `transcript`, `exon`, `CDS`, ...), skipping comment lines:

```
grep -v "^#" annotation.gtf | cut -f3 | sort | uniq -c | sort -rn
```

```
7507 exon
7127 transcript
7127 gene
6913 CDS
6601 start_codon
6600 stop_codon
  4 five_prime_utr
```

The first number comes from `uniq -c`, the value beside it. This same four-step sequence, `cut` → `sort` → `uniq -c` → `sort -rn`, applies to any column of any table, and is the most frequently used pipeline in the whole course.

Reading the numbers. The annotation describes genes and their parts. A gene is transcribed into one or more transcripts; each transcript is made of exons; the protein-coding part is the CDS. There are more exons than transcripts because a transcript is usually built from several exons, so the counts above are exactly what the biology predicts.

2.4.5 The other filters

- `tr` replaces or deletes single characters: `tr 'a-z' 'A-Z'` (to upper case), `tr -d '\r'` (strip the carriage-return character `\r` that Windows adds to line ends), `tr ':' '\n'` (turn colons into newlines).

- `comm` compares **two sorted** lists and prints three columns: lines only in the first, only in the second, and in both. `comm -12 a b` shows only the lines they share.
- `paste a b` joins files side by side (as columns), separated by a tab.
- `column -t` aligns columns to a common width so the result is easy to read (for viewing, not for further processing).

A worked example with `comm`: the data package has two gene lists, and we want the genes common to both. `comm` requires its inputs sorted, which we supply on the fly:

```
comm -12 <(sort genes_set_a.txt) <(sort genes_set_b.txt) | wc -l
```

20

The construct `<(sort ...)` is **process substitution**: it lets a command's output stand in for a file name. It is explained further in Chapter 7; here it simply feeds two sorted lists to `comm` without creating temporary files.

2.5 Searching with `grep` and regular expressions

`grep` prints the lines of a file (or stream) that match a pattern. The pattern can be plain text or a **regular expression** (regex), a small language for describing text patterns.

2.5.1 The most useful options

- `-c`, print the count of matching lines, not the lines;
- `-v`, invert: print lines that do **not** match;
- `-o`, print only the match, not the whole line (useful with `-E`);
- `-i`, ignore case (`chr` also matches `Chr` and `CHR`);
- `-n`, prefix each result with its line number;
- `-w`, match whole words only;
- `-E`, enable extended regular expressions (see below).

```
grep -c "^>" transcripts.fa           # how many FASTA headers
grep -v "^#" annotation.gtf          # skip comment lines
grep -in "error" run.log              # find "error", any case, with line
↪ numbers
```

The first of these, on the course transcriptome, prints 7036, the number of sequence records in the file.

2.5.2 Elements of a regular expression

- `^` and `$` are **anchors**. They match no character but a *position*: `^` is the start of the line, `$` its end. So `^>` means “the line begins with `>`”, and `x$` means “the line ends in `x`”. Without an anchor, a pattern matches anywhere in the line.
- `.`, any single character.



Figure 2.3: Anatomy of a regular expression. The pattern `^chr[0-9]+` reads: at the start of the line, the literal text `chr`, followed by one or more digits.

- `[0-9]`, one digit; `[a-z]`, one lower-case letter; `[A-Za-z]`, one letter. Sets combine: `[A-Za-z0-9]`.
- `[^...]`, any character **not** in the set (a caret at the start of the brackets negates it).
- Quantifiers set how many times the previous item repeats: `*` (zero or more), `+` (one or more), `?` (zero or one), `{2}` (exactly two), `{2,4}` (two to four).
- `\b`, a word boundary; `|` (in `-E` mode), alternation, “either/or”.

2.5.3 BRE and ERE: two dialects

By default `grep` uses *basic* regular expressions (BRE), in which `+`, `?`, `|` and `{}` must be preceded by a backslash. It is far more convenient to add `-E` (*extended*, ERE), where they work directly. Use `grep -E` whenever you need `+`, `{n}` or `|`.

```
grep -E "^chr[0-9]+" file.bed          # "chr" and at least one digit, at line
↪ start
grep -E "gene|exon" annotation.gtf    # lines containing "gene" OR "exon"
grep -oE "[0-9]+" file                # print each run of digits on its own line
```

A worked example that combines `grep -oE` with the count-and-rank pattern, the gene biotypes recorded in the annotation’s attribute column:

```
grep -oE 'gene_biotype "[^"]+"' annotation.gtf | sort | uniq -c | sort -rn |
↪ head
```

```
40231 gene_biotype "protein_coding"
957 gene_biotype "tRNA"
273 gene_biotype "transposable_element"
233 gene_biotype "snoRNA"
77 gene_biotype "rRNA"
```

Mind the dot. In a regex, `.` means “any character”, not a literal dot. To match an actual dot, in a file extension, say, escape it: `\.fa`.

Regular expressions are best practised interactively; the course page links a regex trainer.

2.6 Compression and archives

Sequence files can be gigabytes in size, so in bioinformatics they are kept compressed, most often in **gzip** format (the `.gz` extension). The key principle: do not unpack them to disk without need, process them **as a stream**, decompressing on the fly and passing the data straight through a pipeline.

- `zcat file.gz | ...`, like `cat`, but decompresses on the fly (on macOS, `gzcat`; portably, `gunzip -c`);
- `zgrep pattern file.gz`, `zless file.gz`, the `.gz` counterparts of `grep` and `less`;
- `gzip -t file.gz`, verify the archive's integrity without unpacking it;
- `gzip file` / `gunzip file.gz`, compress / decompress to disk (when you really need the plain file).

An archive in **tar** format (`.tar`, often compressed as `.tar.gz`) is separate: it packs many files and directories into one. **tar** does not unpack a single member in isolation. It handles the whole directory structure:

```
tar -czf package.tar.gz directory/  # pack a directory (c create, z gzip, f
↪ file)
tar -tzf package.tar.gz             # list the contents without extracting (t)
tar -xzf package.tar.gz             # extract (x)
```

Compression is substantial: the course transcriptome shrinks from 11 MB to about 3.7 MB. You can confirm an archive and preview it without ever writing the plain file to disk:

```
gzip -t transcripts.fa.gz && echo "archive OK"
zcat transcripts.fa.gz | head -2
```

2.7 Common mistakes

Worth reviewing before you hand in the exercises:

1. `uniq` without a preceding `sort`: it counts only *adjacent* duplicates.
2. Forgetting the quotes around `"$var"` when a value contains spaces.
3. Confusing globbing (`*` in file names) with regular expressions (`.*` in `grep`).
4. Forgetting that errors go to `stderr`: `cmd > file` does not capture them.
5. `sort` without `-n` on numbers: alphabetical order puts 10 before 2.
6. `>` instead of `>>`: the first overwrites the file, the second appends. It is easy to erase a result this way.
7. A pattern matched against the wrong case or a different chromosome name (`chr1` vs `Chr1` vs `I`): always check the file first with `head`.

2.8 Command summary

Task	Command
Preview an expansion	<code>echo pattern</code>
Count file lines / matches	<code>wc -l file · grep -c pattern file</code>
Redirect output / append	<code>cmd > file · cmd >> file</code>
Redirect / discard errors	<code>cmd 2> err.log · cmd 2>/dev/null</code>
Build a pipeline	<code>cmd1 \ cmd2 \ cmd3</code>
Select columns	<code>cut -f1,3 file</code>

Task	Command
Sort (numeric / by column)	<code>sort -n · sort -k2,2n</code>
Count and rank	<code>cut -fN \ sort \ uniq -c \ sort -rn</code>
Compare sorted lists	<code>comm -12 a b</code>
Search with a regex	<code>grep -E "pattern" file</code>
Decompress on the fly	<code>zcat file.gz \ ...</code>
Verify / list an archive	<code>gzip -t file.gz · tar -tzf pkg.tar.gz</code>

2.9 Exercises

The exercises use the course data package (`coding_lab_data.zip`). Work in the unpacked directory.

2.1 Using `grep` and `wc` (or a single `grep` option), count how many sequence records are in `transcripts.fa`. Then count how many lines the whole file has. Why is the second number larger?

2.2 Write a pipeline that prints, for `annotation.gtf`, how many features of each type there are, ordered from most to least common. Skip the comment lines.

2.3 From `expression.tsv`, print just the gene names and the three stress columns. (Look up which columns those are with `head -1`.)

2.4 How many features does the annotation have on chromosome IV? (The chromosome is in column 1. Be careful to match the whole field, not a substring.)

2.5 Find the genes common to `genes_set_a.txt` and `genes_set_b.txt`, and the genes that appear only in `genes_set_a.txt`. How many are there in each case?

2.6 Redirect the feature-type counts from exercise 2.2 into a file called `feature_counts.txt`, and at the same time display them on screen, using `tee`. Confirm the file was written.

Chapter 3

The VIM text editor

So far you have only *looked* at files, with `cat`, `less` and `head`. To write a script or correct a configuration file you need an **editor** that runs inside the terminal, with no separate window. This matters above all when you work on a remote machine or a compute cluster: you connect over `ssh`, and often the only editor available is one that runs in the console. This chapter teaches **VIM**, the editor you will find on essentially every Unix system.

Why learn VIM specifically? Two reasons. First, it is *everywhere*: no matter how unfamiliar the server, `vim` is almost certainly installed. Second, it is often the **default** editor that other tools open for you: `git commit` and `crontab -e` both drop you into it. Without knowing even how to leave VIM, it is easy to get stuck. A small, deliberate set of commands is enough to be productive.

3.1 The modal model

VIM has an unusual way of working, built on **modes**. In most editors every key you press inserts a character. In VIM the keyboard does different things depending on which mode you are in. The one that surprises newcomers is that, immediately after opening a file, you are *not* able to type text: you are in **Normal mode**, where the keys are commands.

The three modes you need are:

- **Normal mode**, the mode VIM starts in. Keys are *commands*: they move the cursor, delete text, paste, and so on. You return here with `Esc`, and you spend most of your time here.
- **Insert mode**, ordinary typing. You enter it from Normal mode with `i` (and a few relatives), type text as in any editor, and leave it with `Esc`.
- **Command-line mode**, for commands typed at the bottom of the screen, beginning with `:`. This is where you save (`:w`), quit (`:q`), and run substitutions.

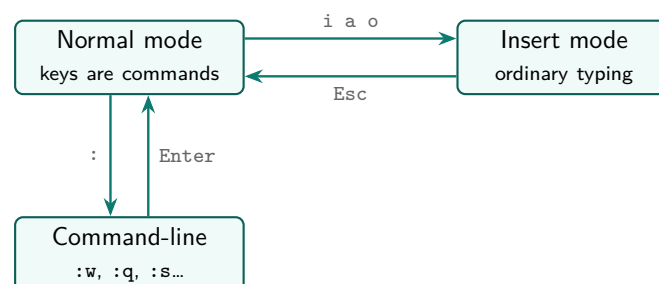


Figure 3.1: VIM's modes. You start in Normal mode, where keys are commands. Press `i` (or `a`, `o`) to enter Insert mode and type text; press `Esc` to return. A leading colon drops into Command-line mode for `:w`, `:q` and similar.

The single most important habit is this: **when in doubt, press Esc**. It always returns you to Normal mode, from where every command is available.

3.2 The VIM screen

Open a file by naming it on the command line (if it does not exist, VIM will create it when you save):

```
vim genes.bed
```

The screen fills with the file's contents. Lines below the end of the file are marked with a ~, which is *not* part of the file: it simply means “there is no line here”. The bottom line is the **status line**: it shows the file name, whether there are unsaved changes, and the cursor's position. When you are in Insert mode, VIM prints -- INSERT -- there, so you can always tell which mode you are in.

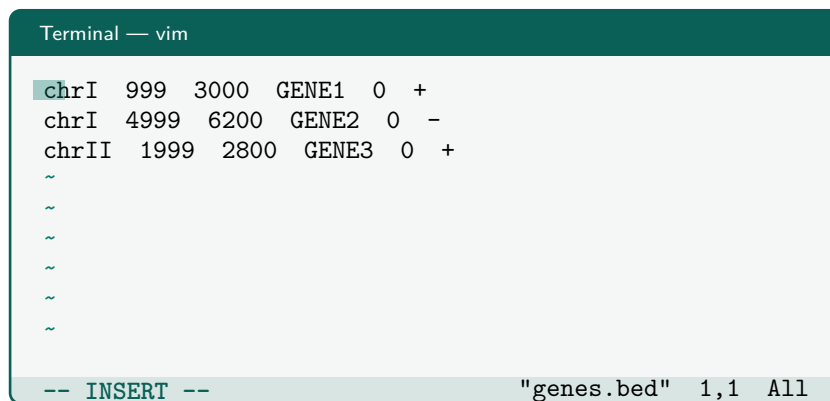


Figure 3.2: A schematic of the VIM screen while editing *genes.bed*. The file's three lines are shown; the teal block is the cursor; the ~ markers stand for lines beyond the end of the file. The status line at the bottom shows the mode (-- INSERT --), the file name, and the cursor position (line, column).

3.3 Surviving: open, insert, save, quit

If you learn nothing else, learn this handful. It is enough to open a file, make a change, and leave safely.

- **i**, enter Insert mode (begin typing before the cursor);
- **Esc**, return to Normal mode;
- **:w**, write (save); **:q**, quit; **:wq**, save and quit;
- **:q!**, quit and **discard** changes (the escape hatch when something has gone wrong);
- **u**, undo; **Ctrl-r**, redo.

The common panic, “I can’t type / I can’t get out”, is almost always because you are in the wrong mode. Press **Esc**, then **:q!** if you want to abandon the file, or **:wq** to keep your changes.

3.4 Moving around in Normal mode

In Normal mode the cursor moves with commands rather than only the arrow keys. The arrow keys do work, but the lettered motions are faster and work even on keyboards or connections where the arrows misbehave.

Keys	Movement
<code>h j k l</code>	left, down, up, right
<code>w / b</code>	forward / back one word
<code>0 / \$</code>	start / end of the line
<code>gg / G</code>	first / last line of the file
<code>:42</code>	jump to line 42
<code>Ctrl-f/Ctrl-b</code>	forward / back one screen

Many commands take a **count** in front: `5j` moves down five lines, `3w` three words forward. This idea of a number followed by a command recurs throughout VIM.

3.5 Editing text

The real speed of VIM comes from combining an **operator** with a **motion**. An operator says *what to do* (delete, change, copy); a motion says *over what*. The combination reads almost like English.

Entering Insert mode:

- `i`, insert before the cursor; `a`, append after it;
- `o`, open a new line below and start typing; `O`, a new line above.

Deleting, copying and pasting (in Normal mode):

- `x`, delete the character under the cursor;
- `dd`, delete (cut) the whole line; `dw`, delete to the end of the word;
- `yy`, yank (copy) the line; `p`, paste after the cursor, `P` before it;
- `cw`, change a word: delete it and drop straight into Insert mode.

The operator-motion grammar generalises: `d$` deletes to the end of the line, `d G` deletes to the end of the file, `3dd` deletes three lines. Learn a few operators (`d`, `y`, `c`) and a few motions (`w`, `$`, `G`), and their combinations give you far more than the sum of the parts.

The undo tree is your safety net. Every change can be undone with `u`, and re-applied with `Ctrl-r`. Because experimentation is cheap, the fastest way to learn these commands is to open a throwaway copy of a file and try them.

3.6 Searching and substitution

To search, press `/`, type a pattern (an ordinary regular expression, as in Chapter 2), and press Enter. `n` jumps to the next match, `N` to the previous. Searching backwards uses `?` instead of `/`.

Substitution is done in Command-line mode with `:s`. The pattern is a regular expression and the general form is `:s/old/new/`:

```
:s/old/new/      substitute the first match on the current line
:s/old/new/g     every match on the current line (g = global on the line)
:%s/old/new/g    every match in the whole file (% = all lines)
:%s/old/new/gc   as above, but ask for confirmation at each (c)
```

For example, to rename every occurrence of the chromosome `chrI` to `I` throughout a BED file (a common adjustment when two files use different naming conventions), you would type `:%s/chrI/I/g`. Because `.` is a regex metacharacter, remember to escape a literal dot as `\.` inside the pattern.

3.7 Working with files and a first configuration

A few more Command-line commands handle the file itself:

- `:w name`, save under a different name (like “save as”);
- `:e name`, open another file for editing;
- `:w` then `:e!`, reload the file from disk, discarding unsaved changes.

VIM reads its settings from a file called `.vimrc` in your home directory. A small one makes the editor far more comfortable from the start:

```
syntax on          " colour-code the file by its type
set number         " show line numbers
set tabstop=4      " a tab is shown as four spaces wide
set expandtab       " insert spaces when Tab is pressed
set autoindent     " keep the indentation of the previous line
set hlsearch       " highlight all matches of a search
```

Lines after a `"` are comments. You do not need to understand every option now; copy this file, and adjust it as your preferences form.

3.8 nano: the simple alternative

When VIM’s model feels like too much, **nano** is the gentle option. It behaves like an ordinary editor (you just type), and it shows its shortcuts along the bottom of the screen at all times, so there is nothing to memorise. In the shortcut list, `^` means the Ctrl key:

```
nano analyse.sh      # open (or create) a file
```

- **Ctrl-O**, write out (save); confirm the name with Enter;
- **Ctrl-X**, exit (nano asks whether to save if there are unsaved changes);
- **Ctrl-K** / **Ctrl-U**, cut / paste a line;
- **Ctrl-W**, search within the file.

For most of the work in this course nano is entirely sufficient. Even if you choose it day to day, it is worth knowing **Esc** and **:q!** in VIM, because VIM is the editor that will sometimes open whether you chose it or not.

3.9 Common mistakes

1. **Typing in Normal mode.** If letters are not appearing (or are triggering odd behaviour), you are in Normal mode, press **i** to insert.
2. **Stuck and unable to quit.** Press **Esc** first, then **:q!** to leave without saving, or **:wq** to save and leave.
3. **:q refuses to quit.** There are unsaved changes. Use **:wq** to save them or **:q!** to discard them.
4. **A substitution changed only one line.** **:s** acts on the current line only; use **:%s/old/new/g** for the whole file.
5. **A forgotten g flag.** **:s/old/new/** changes only the first match on the line; add **g** for all of them.

3.10 Command summary

Task	Keys / command
Open a file	vim file
Insert / append text	i · a · o
Back to Normal mode	Esc
Save / quit / both	:w · :q · :wq
Quit discarding changes	:q!
Undo / redo	u · Ctrl-r
Move by word / line ends	w b · 0 \$
File start / end / line N	gg · G · :N
Delete char / word / line	x · dw · dd
Copy / paste a line	yy · p
Search / next / previous	/pattern · n · N
Substitute in whole file	:%s/old/new/g

3.11 Exercises

These exercises use the course data package. Make a copy of a file before editing it, so the originals stay intact, for example `cp formats/genes.bed my_genes.bed`.

3.1 Open `my_genes.bed` in VIM. Without consulting notes, add a new line at the end describing a fourth gene, then save and quit. Reopen the file with `cat` to confirm your line is there.

3.2 In `my_genes.bed`, use a single substitution command to change every `chrI` and `chrII` to `I` and `II` (that is, remove the `chr` prefix throughout the file). Verify the result.

3.3 Practise moving: open the file, jump to the last line with `G`, back to the first with `gg`, and to line 2 with `:2`. Delete the second line with `dd`, then undo it with `u`.

3.4 Create a `.vimrc` in your home directory containing at least `set number` and `syntax on`. Open any file and confirm that line numbers now appear.

3.5 Copy `expression.tsv` to `my_expr.tsv`, open it in VIM, and use `:%s` to rename the column header `control_1` to `ctrl_1`. Save and check with `head -1 my_expr.tsv`.

Chapter 4

Biological data: databases and formats

Until now the files in this book have been treated as plain text, lines and columns to be counted, filtered and sorted. This chapter is about the **specific formats** in which biological data is actually stored, and about **where that data comes from**. Each format is a convention: a fixed way of writing down sequences, alignments, variants or genomic intervals so that any tool can read them. Knowing the conventions is what lets you reach for the right command, and avoid the mistakes that quietly corrupt an analysis.

A note on tools. The text formats in this chapter (FASTA, FASTQ, SAM, VCF, GTF, BED) are plain text, so the commands you already know (`grep`, `awk`, `cut`, `sort`) work on them directly, and the outputs shown here were computed on the course data. The dedicated tools for these formats (`samtools`, `bcftools`, `bedtools`) are *not* part of the system; they are installed from the **Bioconda** channel (Chapter 7). For those we give the commands exactly as the official documentation defines them, and you run them yourself in the course environment.

The course data package contains two kinds of files. The full-genome files, `transcripts.fa` and `annotation.gtf`, come from Ensembl and use its chromosome names (I, II, ... XVI). The small files under `formats/` are **purpose-built examples**, just a few records each, with simplified names (`chrI`, `chrII`) so that every field is easy to see. We use the examples to read the formats and the full files to compute on real data.

4.1 Where data comes from: databases and accessions

Before any analysis, the data has to be obtained, almost always from a public database. Four repositories cover most needs:

- **Ensembl:** genomes, transcriptomes and annotation for model and farmed species. Files are served over FTP in a predictable layout, `release-<NNN>/<type>/<species>/`, where `<type>` is `fasta`, `gtf`, `gff3` and so on. The yeast genome, for instance, lives under `release-110/fasta/saccharomyces_cerevisiae/dna/`.
- **NCBI:** the largest repository. Genome data is downloaded conveniently with the `datasets` tool (`datasets download genome accession GCF_000146045.2`); individual records also come through Entrez (Chapter 8).
- **UCSC Genome Browser:** genomes and annotation tracks, many in **BED** format, under `goldenPath/<genome>/`, with a set of command-line *UCSC utilities*.
- **SRA (Sequence Read Archive):** raw sequencing reads, fetched with `sra-tools: prefetch SRRXXXXXXX` to download, then `fasterq-dump SRRXXXXXXX` to convert to FASTQ.

Every database issues **accession numbers**, stable identifiers for a specific version of a dataset (GCF_000146045.2, SRR1234567). The accession *together with the database release number* is the minimum you must record in a methods section for the analysis to be reproducible.

4.2 Downloading and verifying files

To fetch a file by URL, use **wget**. The **-c** option resumes an interrupted download and **-P** sets the destination directory:

```
wget -c -P data/ https://ftp.ensembl.org/pub/release-110/gtf/saccharomyces_-  
cerevisiae/Saccharomyces_cerevisiae.R64-1-1.110.gtf.gz
```

For many files, or to synchronise whole directories, **rsync** is better. It transfers only what has changed and resumes cleanly:

```
rsync -av rsync://ftp.ensembl.org/ensembl/pub/release-110/fasta/saccharomyces_-  
cerevisiae/dna/ data/
```

Both print live progress that depends on the network, so only the commands are shown here.

A download can truncate or corrupt a file, and this must be caught *before* analysis. Databases publish **checksums**, short digests that will not match for a damaged file. The most common is MD5:

```
md5sum formats/genes.bed formats/variants.vcf
```

```
011c7288937f4e16dea3d200c7fb9d09  formats/genes.bed  
c49fad13319676fa3784313829a7d3cd  formats/variants.vcf
```

(On macOS the equivalent is **md5 -r**; the digest itself is identical.) A database ships a file of expected sums, and **md5sum -c** reports OK or FAILED for each entry, and exits non-zero on any mismatch, so a script can act on it (Chapter 6).

Files compressed with **gzip** carry a **built-in checksum** as well, which **gzip -t** (test) verifies without unpacking:

```
gzip -t transcripts.fa.gz && echo "archive OK"
```

```
archive OK
```

This is the cheapest way to be sure a downloaded **.gz** is intact before you start.

4.3 FASTA: sequences

FASTA is the simplest sequence format. Each record is a **header** line beginning with **>** (an identifier and a free-text description) followed by one or more **sequence** lines. The first “word” of the header, up to the first space, is the identifier; the rest is description. Here is the first record of **transcripts.fa**:

```
>YPL071C_mRNA cdna chromosome:R64-1-1:XVI:420048:420518:-1 gene:YPL071C ...
ATGAGTTCCCGTTTGAAGAAGTAATGGCAATCCCAACCACATTAGGAAAAGAAATCAT
...
```

The number of sequences is the number of headers (Chapter 2):

```
grep -c "^>" transcripts.fa
```

```
7036
```

Length and G+C content, a basic property of a sequence, are computed per record with **awk**. Because a FASTA sequence is often **wrapped** across several lines, the lines must be joined before measuring; the following accumulates the sequence until the next header. For the first transcript:

```
awk '/^>/{if(name)exit; name=substr($1,2); next} {seq=seq $0}
      END{len=length(seq); g=gsub(/[GCgc]/,"",seq);
          printf "%s\tlen=%d\tGC=%.1f%%\n", name, len, 100*g/len}'
↪ transcripts.fa
```

```
YPL071C_mRNA    len=471 GC=35.7%
```

Mind the line-wrapping. If you measure `length($0)` on a single line of a wrapped FASTA, you get the length of that *line*, not of the sequence. Always join the sequence lines of a record first, or use a dedicated tool such as **seqkit stats**.

4.4 FASTQ: reads with quality

FASTQ stores raw sequencing reads together with a **quality** score for every base. Each record is exactly **four lines**: an identifier beginning with `@`, the sequence, a `+` separator, and the quality string, one character per base.

one record	{	@read_1 instrument:MiSeq	1. identifier (starts with @)
		ATGACTGCCGACCTTAGGCTGATCGGATCC	2. sequence
		+	3. separator (+)
		IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII	4. quality, one char per base

Figure 4.1: The four lines of a FASTQ record, from *formats/reads.fastq*. The quality line has exactly as many characters as the sequence line; each encodes the confidence in the base above it.

The quality line encodes a **Phred score** per base. The numeric value is the character's ASCII code minus 33 (the **Phred+33** encoding, standard today). The character `I` is ASCII 73, so $Q = 73 - 33 = 40$; the character `#` is $Q = 2$. A higher Q means a lower error probability: $Q = 40$ is about one error in 10,000, $Q = 20$ one in 100. Because each record is four lines, the **number of reads is the line count divided by four**:

```
awk 'END{print "reads:", NR/4}' formats/reads.fastq
```

```
reads: 3
```

In the example file the first read is I throughout (all Q = 40), while the second ends in , and # (low quality at the end), the typical picture for Illumina data, where quality falls in the last cycles. Quality filtering of FASTQ is done with dedicated tools (`fastp`, `trimmomatic`) or programmatically (Chapter 8).

4.5 Working with compressed files

Data from databases is nearly always `gzip`-compressed (the `.gz` extension), which saves both space and transfer time. There is no need to unpack it to look inside: `zcat` writes the decompressed contents to standard output, so it drops straight into a pipeline.

```
zcat transcripts.fa.gz | grep -c "^>"
```

```
7036
```

The compression is substantial: for `transcripts.fa`, from about 11.4 MB to 3.7 MB, a factor of roughly three. `zgrep` (search) and `zless` (paging) are the `.gz` counterparts of `grep` and `less`, so the whole toolkit of the previous chapters applies to compressed files without an intermediate copy on disk. (On macOS, `zcat` expects a `.Z` file; use `gzcat` instead.)

Do not unpack “just in case”. `zcat ... | ...` processes a compressed file on the fly; unpacking multi-gigabyte files to disk only to read them wastes space.

4.6 SAM/BAM: alignments

After reads are **aligned** to a reference genome, their positions are stored in **SAM** (text) or its binary equivalent **BAM**. A SAM file has a **header** (lines beginning with `@`: `@HD` for version and sort order, `@SQ` for each reference sequence and its length) and then one **alignment record** per line, with eleven mandatory tab-separated fields:

#	Field	Meaning
1	QNAME	read name
2	FLAG	bitwise flag describing the alignment (see below)
3	RNAME	reference sequence (chromosome), or *
4	POS	start position (1-based)
5	MAPQ	mapping quality
6	CIGAR	alignment description (30M = 30 matched positions)
7 to 9	RNEXT/PNEXT/TLEN	Next read information

#	Field	Meaning
10	SEQ	read sequence
11	QUAL	base qualities (as in FASTQ)

The **FLAG** field is a number whose individual **bits** record facts about the read. The most important is the bit of value 4, *read unmapped*. Since SAM is text, we can test that bit with **awk** to show the structure:

```
awk '!/^@/{ if(int($2/4)%2) unmapped++; else mapped++ }
      END{print "mapped="mapped"\tunmapped="unmapped}' formats/alignment.sam
```

```
mapped=4      unmapped=1
```

The record `read_4` has `FLAG=4` and `RNAME=*`, so it is unmapped; the other four map to `chrI/chrII`. In practice you never decode flags by hand: **samtools** does it, knowing the meaning of every bit. BAM is binary, so it is **never** viewed with **cat**. Use **samtools view**. The core sub-commands (syntax per the htslib documentation) are:

Command	Action
<code>samtools view -b in.sam > out.bam</code>	convert SAM → BAM (<code>-b</code> = BAM output)
<code>samtools view -c in.bam</code>	count alignment records
<code>samtools view -q 30 in.bam</code>	keep only alignments with mapping quality ≥ 30
<code>samtools sort -o sorted.bam in.bam</code>	sort by coordinate
<code>samtools index sorted.bam</code>	build the <code>.bai</code> index for region queries
<code>samtools flagstat in.bam</code>	summarise flags (mapped, paired, ...)

The usual order of work is **view** → **sort** → **index**: many operations (region queries, genome browsers) require a sorted, indexed file.

4.7 VCF: variants

VCF (Variant Call Format) records differences between a sample and the reference (variants). It has a **header**, **##** lines of metadata (format version, reference, definitions of the **INFO** and **FILTER** fields) and one **#CHROM...** line of column names, followed by **variant records**. The eight mandatory columns are **CHROM POS ID REF ALT QUAL FILTER INFO**:

```
#CHROM POS ID REF ALT QUAL FILTER INFO
chrI 1100 . A G 60 PASS DP=35
```

That record: on `chrI`, position 1100 (1-based), an `A→G` substitution, variant quality `QUAL=60`, filter status `PASS`, read depth `DP=35`. Because the header starts with `#`, a pattern of `!/^#/` skips it. Variants per chromosome:

```
grep -v "^#" formats/variants.vcf | cut -f1 | sort | uniq -c
```

```
4 chrI
2 chrII
```

Variants that passed the filter and have quality at least 20:

```
awk -F'\t' '!/^#/ && $6>=20 && $7=="PASS" {print $1, $2, $4">"$5, "QUAL="$6}'
↪ formats/variants.vcf
```

```
chrI 1100 A>G QUAL=60
chrI 2300 G>A QUAL=95
chrI 5550 T>C QUAL=42
chrII 250 A>ATG QUAL=33
```

Two variants with `QUAL<20` and filter `q20` were dropped. The record `chrII:250 A>ATG` is an **insertion** (ALT longer than REF). VCF records indels this way too, not only single-base substitutions. The dedicated tool is **bcftools**, which *understands* the format (its filter expressions work regardless of column order):

Command	Action
<code>bcftools view -i 'QUAL>=20' in.vcf</code>	keep variants matching an expression
<code>bcftools view -f PASS in.vcf</code>	keep only PASS variants
<code>bcftools stats in.vcf</code>	statistics (SNPs, indels, Ts/Tv, ...)
<code>bcftools query -f '%CHROM\t%POS\t%REF\t%ALT\n' in.vcf</code>	extract chosen fields

The `awk` filter above, written to show the structure, is in practice just `bcftools view -i 'QUAL>=20' -f PASS`.

4.8 GTF/GFF3: annotation

GTF and **GFF3** describe *where* features (genes, transcripts, exons) lie on the genome. The format is tabular, nine tab-separated columns: **seqname source feature start end score strand frame attribute**. Coordinates are **1-based and inclusive** (as in VCF and SAM). The last column holds attributes, in GTF, key "value"; pairs. Counting features by type (column 3) on the full yeast annotation:

```
grep -v "^#" annotation.gtf | cut -f3 | sort | uniq -c | sort -rn
```

```
7507 exon
7127 transcript
7127 gene
6913 CDS
6601 start_codon
6600 stop_codon
4 five_prime_utr
```

The total exon length of a gene (with 1-based inclusive coordinates, a feature's length is `end - start + 1`; the `gene_id` is pulled from the attribute column with a portable `sub`):

```
awk -F'\t' ' $3=="exon"{
    g=$9; sub(/.*gene_id "/", "",g); sub(/".*/", "",g);
    len=$5-$4+1; sum[g]+=len
} END{for(x in sum) print x"\texons_bp="sum[x]}' annotation.gtf | sort | head
↵ -3
```

The sum of a gene's exon lengths is its **transcript length** (introns excluded), useful, for example, when normalising expression. GFF3 carries the same information but writes attributes differently (`ID=...`; `Parent=...`) and makes the parent-child hierarchy explicit (gene → transcript → exon).

4.9 BED and coordinate systems

BED is the simplest **interval** format. The first three columns are required, **chrom** **start** **end**, and further optional columns include **name**, **score**, **strand**. The crucial difference from GTF/VCF/SAM is the **coordinate system**: BED is **0-based and half-open**, meaning the start counts from zero and the end points *one past* the last base. The payoff is that an interval's length is simply `end - start`, with no `+1`:

```
awk -F'\t' '{print $4"\t"$1":"$2-"$3"\tlen="$3-$2}' formats/genes.bed
```

```
GENE1  chrI:999-3000  len=2001
GENE2  chrI:4999-6200 len=1201
GENE3  chrII:1999-2800 len=801
```

The two conventions describe the *same* stretch of DNA; they only number it differently. The figure below shows a six-base region with both rulers, and the same three-base interval written in each system.

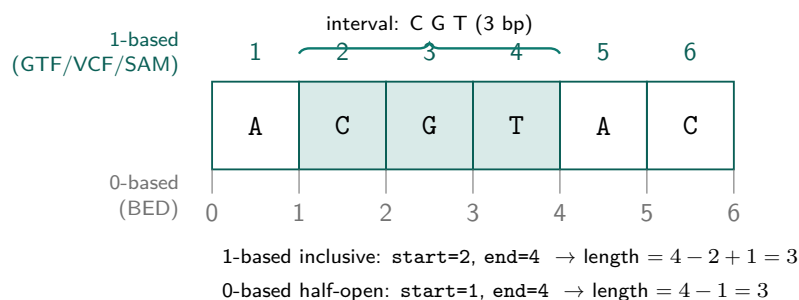


Figure 4.2: The same three-base interval in both coordinate systems. GTF, VCF and SAM number bases from 1 and include both ends; BED numbers the gaps between bases from 0 and excludes the end. Converting a start from GTF to BED subtracts 1; the end is unchanged.

Interval arithmetic is the work of **bedtools**, which uses BED's 0-based half-open coordinates. The general form is `bedtools <sub-command> -a A -b B`, with A and B two interval sets (syntax per the official documentation):

Command	Action
<code>bedtools intersect -a genes.bed -b peaks.bed</code>	parts of A overlapping B
<code>bedtools intersect -a genes.bed -b peaks.bed -wa -u</code>	whole A features with at least one overlap
<code>bedtools coverage -a genes.bed -b peaks.bed</code>	coverage of A by B
<code>bedtools merge -i peaks.bed</code>	merge overlapping/adjacent intervals (sorted input)
<code>bedtools closest -a genes.bed -b peaks.bed</code>	nearest feature in B for each A

In the package, `formats/peaks.bed` is positioned to overlap `formats/genes.bed`, `peak_a` (`chrI:1200-1400`) sits inside `GENE1` (`chrI:999-3000`), so an intersect will report the overlap. **bedtools merge requires sorted input** (`sort -k1,1 -k2,2n`); forgetting this is the most common error, and it fails silently.

4.10 Common mistakes

1. **Analysing before checking integrity.** A download can truncate. Run `md5sum -c` (or `gzip -t` for `.gz`) *before* the analysis, not after the results look strange.
2. **Confusing coordinate systems.** BED is 0-based half-open; GTF/VCF/SAM are 1-based inclusive. Moving coordinates between them without correcting gives an off-by-one error.
3. **Viewing a BAM with cat.** BAM is binary; `cat/less` show noise. Use `samtools view`.
4. **Measuring a wrapped FASTA on one line.** When a sequence spans several lines, `length($0)` is the line length, not the sequence length. Join the lines first.
5. **bedtools merge without sorting.** The input must be sorted by chromosome and position, or the result is silently incomplete.
6. **Forgetting the release number.** An accession without the database release / genome version is not enough to reproduce an analysis.

4.11 Command summary

Task	Command
Download a file (resumable)	<code>wget -c -P data/ URL</code>
Synchronise a directory	<code>rsync -av rsync://... data/</code>
Verify checksums	<code>md5sum -c md5sums.txt</code>
Test a <code>.gz</code> archive	<code>gzip -t file.gz</code>
Process a <code>.gz</code> without unpacking	<code>zcat file.gz ...</code>
Count sequences in FASTA	<code>grep -c ">" file.fa</code>
Count reads in FASTQ	<code>awk 'END{print NR/4}' file.fastq</code>
Count alignments in BAM	<code>samtools view -c file.bam</code>

Task	Command
Sort and index a BAM	<code>samtools sort -o s.bam f.bam && samtools index s.bam</code>
Variants per chromosome	<code>grep -v "^#" f.vcf cut -f1 sort uniq -c</code>
Filter variants	<code>bcftools view -i 'QUAL>=20' -f PASS f.vcf</code>
Features by type (GTF)	<code>grep -v "^#" f.gtf cut -f3 sort uniq -c</code>
Interval length (BED)	<code>awk '{print \$3-\$2}' f.bed</code>
Intersect intervals	<code>bedtools intersect -a A.bed -b B.bed</code>

4.12 Exercises

These exercises use the course data package, the full files in the top directory and the small examples under `formats/`.

4.1 Count the sequences in `transcripts.fa`, then count them again by streaming the compressed copy (`transcripts.fa.gz`) through `zcat` without unpacking it. Confirm the two counts agree.

4.2 For `formats/reads.fastq`, report the number of reads using the four-lines-per-record rule. Then print, for each read, its identifier and its length.

4.3 In `formats/variants.vcf`, list every variant on `chrI` that did **not** pass the filter (i.e. `FILTER` is not `PASS`). How many are there?

4.4 Using `annotation.gtf`, count how many **genes** it contains (feature type `gene` in column 3). Then compute the total exon length of the gene `YDL246C`.

4.5 The gene `GENE3` appears in `formats/genes.bed` as `chrII 1999 2800`. Write its coordinates as they would appear in a GTF file (1-based, inclusive), and verify that both encodings give the same length.

4.6 Verify the integrity of `transcripts.fa.gz` with `gzip -t`. Then compute its MD5 sum with `md5sum` (or `md5 -r` on macOS) and note why you would record that value alongside the Ensembl release number in a methods section.

Chapter 5

The AWK programming language

The text filters of Chapter 2 each do one thing: `cut` takes columns, `sort` orders lines, `grep` keeps matches. Most real questions about a data file, however, mix selecting, computing and formatting in a single step: *print the name and length of every exon, average the third column for each gene, count how many variants passed the quality filter*. **AWK** is a small programming language built exactly for this. It reads a file line by line, splits each line into fields automatically, and lets you say, for any line that matches a condition, what to compute and print. A single line of AWK often replaces a short pipeline, and, unlike a pipeline, it can keep running totals and tables as it goes.

Why learn AWK. Tabular text is the native shape of bioinformatics data: GTF, BED, VCF, SAM and expression tables are all columns separated by tabs. AWK understands this shape without being told: it hands you the columns as `$1`, `$2`, ... and the line as `$0`. For column arithmetic, per-group counting and quick reformatting it is faster to write than a Python script and faster to run than a chain of `cut/sort/paste`. It is installed on every Unix system, so a working AWK one-liner is portable to any cluster. The name is simply the initials of its three authors (Aho, Weinberger, Kernighan); on Linux the program is usually `gawk` (GNU AWK), reached through the name `awk`.

Every example below is computed on the course data: the expression table `expression.tsv`, the annotation `annotation.gtf`, the transcriptome `transcripts.fa`, and the small examples under `formats/`.

5.1 The data model: records and fields

AWK sees a file as a sequence of **records**, and each record as a sequence of **fields**. By default a record is one line, and fields are the pieces separated by whitespace (any run of spaces or tabs). AWK splits every line for you and makes the pieces available by number:

- `$0` is the whole record (the entire line);
- `$1` is the first field, `$2` the second, and so on;
- `$NF` is the **last** field, whatever its number, because `NF` holds the **number** of **fields** on the current line;
- `NR` is the current record **number** (line number), counting from 1.

Figure 5.1 shows the split for one line of `formats/genes.bed`.

Because `NF` is computed for every line, the same program adapts to files with different numbers of columns, and `$NF` always points at the last field even when you do not know how many there are:

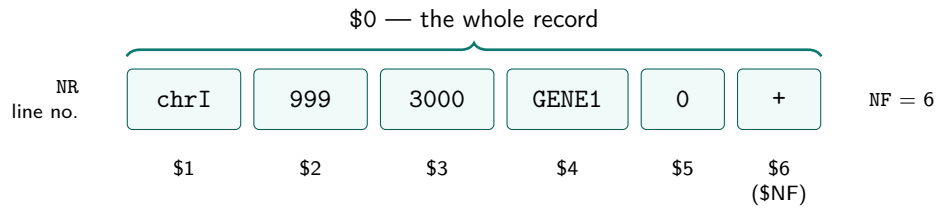


Figure 5.1: AWK splits each line into fields automatically. `$0` is the whole line, `$1...$NF` are the fields, `NF` counts them and `NR` numbers the lines. This is the line `chrI 999 3000 GENE1 0 +` from `formats/genes.bed`.

```
awk '{print "line " NR " has " NF " fields"}' formats/genes.bed
```

```
line 1 has 6 fields
line 2 has 6 fields
line 3 has 6 fields
```

5.2 The pattern-action model

An AWK program is a list of rules, each written as

```
pattern { action }
```

For every input line, AWK tests the **pattern**; if it is true, it runs the **action** (the code in braces). That is the whole model. Either part may be omitted: a pattern with no action prints the matching line (the default action), and an action with no pattern runs on *every* line. Two special patterns bracket the run: **BEGIN** runs once before the first line is read, and **END** runs once after the last line, ideal for printing a header or a final total. Figure 5.2 shows the flow.

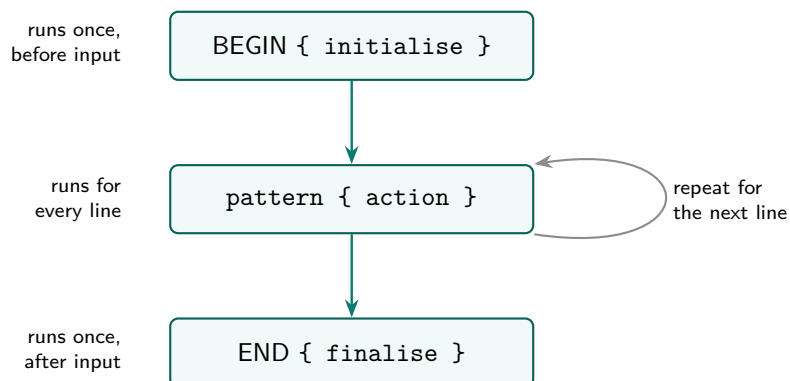


Figure 5.2: The pattern-action model. **BEGIN** and **END** run once; between them AWK reads the input one line at a time and runs each matching rule's action. Most one-liners are a single `pattern { action }` rule.

The simplest useful program is a pattern with the default action. To print only the lines where the first expression value exceeds 1000, the pattern is the comparison and the action is left out:

```
awk -F'\t' 'NR>1 && $2>1000 {print $1, $2}' expression.tsv
```

```
YBR118W 2011
YAL003W 1567
YHR174W 3450
YKL060C 1203
YFL039C 2890
YPL240C 1780
YMR186W 1340
YGR192C 4102
```

Here `-F'\t'` sets the field separator to a tab (the file is tab-separated), `NR>1` skips the header line, and `$2>1000` keeps only the rows we want. The comma in `print $1, $2` inserts a space between the fields.

5.3 Running AWK and setting the field separator

An AWK program is given as a single-quoted string; the file name follows. The quoting matters, single quotes stop the shell from touching `$1`, which it would otherwise try to expand as a shell variable.

The default separator (whitespace) is right for files split on spaces or on single tabs, but biological tables are formally **tab-separated**, and a field may itself contain spaces (a gene description, for example). For those, set the separator explicitly. There are two ways, equivalent in effect:

```
awk -F'\t' '...' file           # -F on the command line
awk 'BEGIN{FS="\t"} ...' file   # FS set in a BEGIN block
```

`FS` is the **field separator**; its partner `OFS` is the **output field separator**, the string AWK puts between fields when you print them with commas. To read a GTF (tab-separated) and write a tab-separated result, set both:

```
awk -F'\t' 'BEGIN{OFS="\t"} $3=="gene" {print $1, $4, $5}' annotation.gtf |
↵ head -3
```

```
IV 8683    9756
IV 17577   18566
IV 1248154 1249821
```

Variables can also be passed in from outside with `-v name=value`, which is how you parameterise a one-liner without editing the program text:

```
awk -F'\t' -v col=4 'NR>1 {print $col}' expression.tsv | head -3
```

```
151
2103
1498
```

5.4 Patterns: choosing the lines

The pattern decides which lines an action touches. The useful forms are:

- **Comparisons** on fields or built-in variables: `$3=="exon"`, `$5>1000`, `NR>1`, `NF==9`. Numeric and string comparison are both written with the same operators; AWK compares numerically when both sides look like numbers.
- **Regular-expression match** with `~` (matches) and `!~` (does not match): `$1 ~ /^chr/` is true when the first field starts with `chr`. A bare `/pattern/` tests the whole line `$0`.
- **Logical combinations** with `&&` (and), `||` (or), `!` (not): `$3=="exon" && $5-$4+1 > 1000`.
- **Ranges** between two patterns, written `pattern1, pattern2`, which select every line from the first match to the second (used less often on data files).

For instance, counting the transcript records whose header mentions a particular chromosome is a single regex pattern over `$0`:

```
awk '/chromosome:R64-1-1:XVI:/{n++} END{print n}' transcripts.fa
```

```
536
```

5.5 Computing: arithmetic, sums and printf

Actions are small programs. They have variables (no declaration needed, an unset variable starts at 0 or the empty string), the usual arithmetic, and the built-in functions `length()`, `substr()`, `gsub()` and others. A running total is the classic pattern: add to a variable on every matching line, then print it in `END`. The mean of the first control column, skipping the header, is

```
awk -F'\t' 'NR>1 {s += $2; n++} END {printf "mean control_1 = %.1f (n=%d)\n",  
↪ s/n, n}' expression.tsv
```

```
mean control_1 = 1413.7 (n=15)
```

`print` is fine for quick output, but `printf` controls the format exactly, which matters for tables and for rounding. It uses C-style conversions inside a format string: `%s` for a string, `%d` for an integer, `%.2f` for a float with two decimals, and width specifiers such as `%-10s` (left-justified in 10 columns) or `%8.1f` (8 columns, one decimal). Unlike `print`, `printf` adds no newline of its own. You write `\n` where you want one. A formatted two-column table of each gene's mean expression across all six samples:

```
awk -F'\t' 'NR>1 {m=($2+$3+$4+$5+$6+$7)/6; printf "%-10s %8.1f\n", $1, m}'  
↪ expression.tsv | head -3
```

```
YDL246C      251.2  
YBR118W      1950.5  
YAL003W      1592.8
```

5.6 Associative arrays: counting and grouping

AWK's arrays are indexed by **strings**, not just numbers. This is what makes per-group counting a one-liner. Writing `n[$3]++` uses the value of field 3 as the key and increments the count stored under it; a key that has not been seen starts at zero. After the whole file is read, a `for (key in array)` loop in `END` prints the tally. Counting the feature types in the annotation (column 3 of the GTF):

```
awk -F'\t' ' !/^#/{n[$3]++} END{for (f in n) print n[f], f}' annotation.gtf |
↪ sort -rn
```

```
7507 exon
7127 transcript
7127 gene
6913 CDS
6601 start_codon
6600 stop_codon
4 five_prime_utr
```

The same idea groups by any field. To count genes per chromosome, key the array on field 1 but only for gene lines:

```
awk -F'\t' '$3=="gene"{n[$1]++} END{for (c in n) print c, n[c]}' annotation.gtf
↪ | sort -k2 -rn | head -5
```

```
IV 885
XII 639
VII 639
XV 636
XIII 549
```

The `!/^#/` pattern in the first example skips GTF comment lines (those beginning with `#`); the `sort` afterwards just orders what AWK emits, since a `for...in` loop visits keys in no particular order.

5.7 Putting it together: BEGIN, END and a real statistic

With patterns, arithmetic and `END` in hand, a complete descriptive statistic fits on one line. The mean exon length across the annotation, summing `end - start + 1` over every `exon` line, then dividing in `END`, is

```
awk -F'\t' '$3=="exon" {s += $5-$4+1; c++} END {printf "mean exon length = %.1f
↪ bp (n=%d)\n", s/c, c}' annotation.gtf
```

```
mean exon length = 1219.2 bp (n=7507)
```

Tracking a maximum uses the same shape with a comparison instead of a sum. Keep the largest value seen so far, and the line it came from:

```
awk -F'\t' ' $3=="exon"{L=$5-$4+1; if (L>max){max=L; where=$1":"$4"-"$5}}
↪ END{print "longest exon =", max, "bp at", where}' annotation.gtf
```

```
longest exon = 14733 bp at XII:349006-363738
```

5.8 Recipes for biological data

A few patterns come up constantly when processing sequence and annotation files.

GC content of a sequence. FASTA sequences are wrapped across many lines, so the sequence of a record must first be joined before its bases are counted. The function `gsub(/regex/, "", string)` deletes every match *and returns the number of substitutions it made*, which is exactly the count of matching bases. The length must be measured **before** `gsub` shortens the string:

```
awk '/^>/{if(seq){L=length(seq); t=seq; gc=gsub(/[GCgc]/,"",t); printf "%s
↪ len=%d GC=%.1f%%\n", name, L, 100*gc/L}; name=substr($1,2); seq=""; next}
    {seq=seq $0}
    END{L=length(seq); t=seq; gc=gsub(/[GCgc]/,"",t); printf "%s len=%d
↪ GC=%.1f%%\n", name, L, 100*gc/L}' transcripts.fa | head -1
```

```
YPL071C_mRNA len=471 GC=35.7%
```

The `/^>/` rule fires on each header: it finishes the previous record, then takes the new name from `$1` with the leading `>` stripped by `substr($1,2)` and resets `seq`. The unconditional second rule appends every non-header line to `seq`. The `END` block flushes the last record, which has no following header to trigger it.

Converting GTF to BED. GTF coordinates are 1-based and inclusive; BED starts are 0-based (Chapter 4). The conversion is one subtraction, `start - 1`, with the separators set to tabs on both sides:

```
awk -F'\t' 'BEGIN{OFS="\t"} $3=="gene"{print $1, $4-1, $5}' annotation.gtf |
↪ head -3
```

```
IV 8682    9756
IV 17576   18566
IV 1248153 1249821
```

Fold change per gene. Dividing the mean of the stress samples by the mean of the controls gives a quick per-gene ratio:

```
awk -F'\t' 'NR>1 {ctrl=($2+$3+$4)/3; str=($5+$6+$7)/3; printf "%s\t%.2f\n",
↪ $1, str/ctrl}' expression.tsv | head -4
```

```
YDL246C 2.50
YBR118W 0.92
YAL003W 1.05
YGL008C 4.40
```

The first gene is induced 2.5-fold under stress; the second is essentially unchanged.

5.9 A companion tool: sed

AWK is built around fields and arithmetic. Its natural companion, **sed** (the **s**tream **e**ditor), is built around one job: transforming text as it flows past, most often by **substitution**. Where AWK thinks in columns, **sed** thinks in patterns on whole lines. The two are frequently used together: **sed** to tidy the text, AWK to compute on it.

The core command is `s/pattern/replacement/`, applied to each line:

```
sed 's/^/chr/' annotation_genes.gtf | cut -f1-5 | head -3
```

Prepending `chr` to every line converts Ensembl chromosome names (`IV`) to the UCSC style (`chrIV`), a routine fix when combining data from the two sources. Run on the **gene** lines of the annotation it gives:

```
chrIV    sgf gene      8683      9756
chrIV    sgf gene     17577     18566
chrIV    sgf gene    1248154   1249821
```

Three features cover most real use:

- **Addressing** restricts a command to selected lines. With `-n` (suppress the automatic printing) and the `p` flag, `sed -n '1,2p' file` prints only lines 1 and 2; `/^#/d` deletes every comment line.
- **Captured groups** with `\( \)` (or `( )` under `-E`) remember part of the match for the replacement, referred to as `\1`, `\2`, Pulling the `gene_id` value out of a GTF attribute column:

```
sed -E 's/.*gene_id "([^"]+)".*/\1/' gene_attributes.txt | head -3
```

```
YDL246C
YDL243C
YDR387C
```

- **In-place editing** with `-i` rewrites the file itself instead of printing to the screen. It is destructive, so keep a backup: GNU **sed** writes one with `-i.bak`. (On macOS the flag requires an explicit suffix, `-i ''` for none; see the note below.)

As a rule of thumb: reach for **sed** to clean or rewrite text, and for AWK to select by column and compute. A pipeline that does both (strip comment lines with **sed**, then count features with AWK) is idiomatic Unix.

5.10 Common mistakes

- **Forgetting `-F'\t'` on a tab-separated file with spaces in a field.** With the default separator, a description field containing spaces is split into several fields and every column number after it is wrong. Set `FS` to a tab for GTF, BED, VCF and expression tables.

- **Letting the shell eat \$1.** The program must be in **single** quotes. In double quotes the shell expands \$1 before AWK ever sees it.
- **Measuring length after gsub.** gsub shortens the string it edits, so read `length()` *before* deleting bases, or the GC percentage is computed against the wrong total.
- **Expecting for (k in a) to be ordered.** Associative arrays have no inherent order; pipe the output through `sort` if you need one.
- **Confusing = and ==.** A single = inside a pattern is an assignment and is always true; comparison needs ==.
- **sed -i on macOS.** BSD `sed` (the default on macOS) requires a suffix argument: `sed -i '' 's/.../.../' file` edits in place with no backup, whereas GNU `sed` on Linux accepts a bare `-i`. Write scripts for the Linux form and note the difference when testing on a Mac.

5.11 Command summary

Construct	Meaning
\$0, \$1, \$NF	whole line, first field, last field
NR, NF	record (line) number; number of fields
FS, OFS	input / output field separator
-F'\t'	set input separator on the command line
-v n=val	pass a variable into the program
pattern { action }	run <i>action</i> on lines matching <i>pattern</i>
BEGIN{ } / END{ }	run once before / after all input
\$3=="exon"	comparison pattern
\$1 ~ /^chr/	regular-expression match
a[\$1]++	count by key (associative array)
printf "%-10s %.2f\n", s, x	formatted output
length(s), substr(s,i), gsub(/re/,"",s)	string functions
sed 's/a/b/'	substitute <i>a</i> with <i>b</i> on each line
sed -n '1,5p'	print only lines 1 to 5
sed -E 's/..(..)\1/'	capture group, reused as \1
sed -i.bak 's/a/b/' f	edit file in place (GNU; keep backup)

5.12 Materials

- *The GNU Awk User's Guide*, the complete gawk manual, freely available at <https://www.gnu.org/software/gawk/manual/>.
- *Sed & Awk* (Dougherty & Robbins), the standard reference for both tools.
- Eric Pement's AWK and sed one-liner collections, widely mirrored online, are a compact catalogue of practical patterns.
- [explainshell.com](https://www.explainshell.com) breaks any command line, including AWK and sed invocations, into its documented parts.

5.13 Exercises

The exercises use the course data package. Copy the files you need into a working directory first.

Exercise 5.1. Using `expression.tsv`, print the gene name and the value of `stress_1` (column 5) for every gene whose `stress_1` exceeds 2000. Skip the header line. (*Hint: a comparison pattern with `NR>1`.*)

Exercise 5.2. Compute the mean of column `control_2` over all genes, printed to one decimal place with `printf`. Then modify your program to print the mean of *every* numeric column by keeping one running sum per column in an array.

Exercise 5.3. From `annotation.gtf`, count how many features of each type (column 3) exist, excluding comment lines, and sort the result by count. Confirm that the number of `gene` features matches the number of `transcript` features for this organism, and explain in one sentence why they might differ in another.

Exercise 5.4. Write a one-liner that converts the `exon` lines of `annotation.gtf` to three-column BED (`chrom`, `start`, `end`) with tab output, remembering the 1-based→0-based start conversion. Count the output lines and check the number against Exercise 5.3.

Exercise 5.5. Using `transcripts.fa`, adapt the GC-content program so that it prints only transcripts shorter than 300 bases, with their length and GC percentage. (*Hint: add a length test before the `printf`.*)

Exercise 5.6. The chromosome names in `annotation.gtf` are bare Roman numerals (`IV`), but `formats/genes.bed` uses the `chrI` style. Use `sed` to rewrite the GTF chromosome column to the `chr`-prefixed form, then pipe the result through AWK to confirm with `a[$1]++` that the renamed names now begin with `chr`. Describe what would go wrong if you applied the same `sed 's/^/chr/'` to `formats/genes.bed`.

Chapter 6

Bash as a scripting language

A single command in a pipeline solves one problem once. The moment the same work has to be done on a dozen files, repeated next week, or handed to a colleague, it belongs in a **script**, a text file holding a sequence of commands that runs like a small program of your own. Everything in the previous chapters (redirection, pipelines, **grep**, **cut**, **sed**, **awk**) stays exactly the same inside a script; what a script adds is the surrounding logic: variables that give names to values, conditions that decide what to do, loops that repeat work across many files, and functions that package a step so it can be reused.

Why write scripts. A script gives three things a lone command cannot: **repeatability**, the same file runs the same steps every time, so an analysis can be rerun and checked; **automation**, a loop walks over many files without retyping; and **readability**, comments and sensible variable names record what is happening and why. A shell script is usually the *glue* of a bioinformatics workflow: it joins the filters you already know with the control logic that turns them into a procedure. That is the subject of this chapter.

Working on macOS or Windows? macOS ships an old Bash (version 3.2, for licensing reasons) in which some features, notably associative arrays (**declare -A**), do not work. The course therefore uses a **conda** environment that provides a modern Bash 5.x, so every example behaves the same on any system. On Windows, scripts are written under **WSL** (Linux), not PowerShell, and files saved by a Windows editor may carry `\r` line endings that break scripts (see *Common mistakes*).

6.1 Anatomy of a script

A script is an ordinary text file. Its first line is usually a **shebang**, the characters `#!` followed by the program that should run the rest of the file:

```
#!/usr/bin/env bash
# describe what the script does

echo "Starting analysis"
```

Writing `#!/usr/bin/env bash` is portable: it looks for **bash** on the **PATH**, so it finds the **conda** version too. The alternative `#!/bin/bash` hard-codes a path (on macOS that is the outdated Bash 3.2, which is why **env** is preferred). Any line starting with `#` other than the shebang is a **comment** and is ignored.

There are two ways to run a script:

Invocation	Requirement	Behaviour
<code>bash script.sh</code>	none	<code>bash</code> runs it directly; the shebang is ignored, no <code>+x</code> needed
<code>./script.sh</code>	the file is executable (<code>chmod +x</code>)	the system reads the shebang and picks the interpreter

```
chmod +x script.sh      # make it executable (once)
./script.sh             # run it via the shebang
```

The `./` is required: without it the shell would search the `PATH` for a program named `script.sh` instead of looking in the current directory.

6.2 Variables and quoting

A variable is created by assignment, with **no spaces** around the `=`, and read back with a `$`:

```
sample="control_1"
echo "Column: $sample"      # Column: control_1
```

A space around `=` is the classic beginner's error: `sample = "x"` is read as a command `sample` with arguments `=` and `x`. Hence no spaces on assignment.

A variable's value is **almost always wrapped in double quotes** when used: `"$sample"`. Without the quotes, the shell splits the value on spaces and expands any wildcard characters, a frequent, hard-to-trace source of bugs with file names. The rule: quote your variables unless you deliberately want word-splitting.

The output of a command is captured into a variable with `$( ... )`:

```
count=$(grep -c "^>" transcripts.fa)
echo "Sequences: $count"      # Sequences: 7036
```

6.3 Positional arguments

A script receives data from the command line through **positional arguments**, available inside the script as special variables:

Variable	Meaning
<code>\$1</code> , <code>\$2</code> , ...	first, second, ... argument
<code>\$0</code>	the name of the script itself
<code>\$#</code>	the number of arguments given
<code>\$@</code>	all arguments, as separate words (quote as <code>"\$@"</code>)
<code>shift</code>	drops <code>\$1</code> and renumbers the rest

A loop over all arguments, the usual shape when a script is handed a list of files:

```
for file in "$@"; do
    echo "processing: $file"
done
```

6.4 Transforming file names

The shell can cut pieces out of a variable's value without `sed` or `cut`. This mechanism, **parameter expansion**, is especially handy for file names, stripping a directory, dropping an extension, substituting part of a name:

Form	Effect	For <code>f=formats/genes.bed</code>
<code>\${f##*/}</code>	remove the longest prefix up to / (the bare file name)	<code>genes.bed</code>
<code>\${f%.bed}</code>	remove the suffix <code>.bed</code>	<code>formats/genes</code>
<code>\${f%.*}</code>	remove the longest suffix from the first dot	<code>formats/genes</code>
<code>\${f/old/new}</code>	replace the first <code>old</code> with <code>new</code>	<code>n/a</code>
<code>\${#f}</code>	length of the value	<code>15</code>
<code>\${v:-default}</code>	value of <code>v</code> , or <code>default</code> if unset/empty	<code>n/a</code>

The key distinction: `##` work from the **start** of the value, `%%` from the **end**; a single character removes the shortest match, a doubled one the longest. Combining them turns a full path into a clean sample identifier:

```
f="formats/genes.bed"
echo "${f##*/}"      # genes.bed    (drop the directory)
echo "${f%.bed}"     # formats/genes (drop the extension)
```

```
genes.bed
formats/genes
```

The programs `basename` and `dirname` give the same results and are sometimes clearer:

```
basename formats/genes.bed .bed    # genes
dirname  formats/genes.bed          # formats
```

6.5 Exit status and combining commands

Every command, when it finishes, returns an **exit status**: 0 means success, any non-zero value means failure. The status of the most recent command is in `$?`:

```
grep -q "^>" transcripts.fa      # -q: no output, just set the status
echo $?                          # 0, because headers were found
```

Two operators chain commands on that status:

- `cmd1 && cmd2`, run `cmd2` **only** if `cmd1` succeeded;
- `cmd1 || cmd2`, run `cmd2` **only** if `cmd1` failed.

```
grep -q "^>" transcripts.fa && echo "FASTA headers present"
grep -q "^>" formats/genes.bed || echo "no FASTA headers in genes.bed"
```

```
FASTA headers present
no FASTA headers in genes.bed
```

A script itself ends with a status too. `exit N` stops the script immediately and sets its status to `N` (0 for success, non-zero for an error). The usual use is to bail out early when there is no point continuing, for instance when a required argument is missing:

```
if (( $# == 0 )); then
    echo "usage: $0 <file>" >&2
    exit 1
fi
```

This is the foundation of conditionals: `if` tests exactly this kind of exit status. (Hardening a script more thoroughly, with `set -euo pipefail`, belongs to the next chapter.)

6.6 Conditionals

The `if` statement runs a block when a command returns status 0:

```
if grep -q "^>" "$file"; then
    echo "this is a FASTA file"
else
    echo "no FASTA headers"
fi
```

To test files and compare values, use the `[[ ... ]]` construct. The common file tests:

Test	True when
<code>[[-f "\$p"]]</code>	<code>p</code> is a regular file
<code>[[-d "\$p"]]</code>	<code>p</code> is a directory
<code>[[-e "\$p"]]</code>	<code>p</code> exists (file or directory)
<code>[[-s "\$p"]]</code>	<code>p</code> exists and is non-empty
<code>[[-z "\$s"]]</code> / <code>[[-n "\$s"]]</code>	string <code>s</code> is empty / non-empty

String and numeric comparisons use **different operators**, a frequent source of mistakes:

Kind	Equal	Not equal	Less	Greater
string	=	!=	<	>
number	-eq	-ne	-lt	-gt

```
if [[ -f "$file" && -s "$file" ]]; then
    echo "file exists and is not empty"
fi

n=$(grep -c "^>" "$file")
if (( n > 1 )); then                # numeric comparison in (( ))
    echo "file holds $n sequences"
fi
```

Numeric comparisons read most naturally inside `(( ... ))` with ordinary `>/<`; the `-gt/-lt` forms work inside `[[ ... ]]`. Quoting variables in `[[ ... ]]` guards against errors when a variable is empty. A leading `!` negates a test: `[[ ! -f "$p" ]]` is true exactly when the file does not exist.

When one value must be matched against several possibilities, **case** is clearer than a chain of `if/elif`. It matches a value against **patterns** (globbing-style: `*`, `?`), with `|` separating alternatives in one branch:

```
for f in transcripts.fa formats/genes.bed expression.tsv; do
    case "$f" in
        *.fa|*.fasta) kind="sequences" ;;
        *.bed)        kind="intervals" ;;
        *.tsv|*.csv)   kind="table" ;;
        *)             kind="unknown" ;;      # default branch
    esac
    echo "$f -> $kind"
done
```

```
transcripts.fa -> sequences
formats/genes.bed -> intervals
expression.tsv -> table
```

Each branch ends with `;;`, and the whole statement with `esac` (**case** reversed); the final `*` pattern is the default. Figure 6.1 (left) shows the branching flow.

6.7 Loops

A **for** loop walks over a list of values, most often the files matching a pattern. The counting idiom from earlier chapters, now applied to every BED file in a directory:

```
for f in formats/*.bed; do
  n=$(grep -c . "$f")
  echo "${f##*/}: $n intervals"
done
```

```
genes.bed: 3 intervals
peaks.bed: 4 intervals
```

A C-style numeric variant is useful when a counter is needed:

```
for ((i=1; i<=3; i++)); do
  echo "round $i"
done
```

A `while` loop repeats as long as its condition holds (`until` repeats while it is false). The condition is tested exactly as in `if`:

```
i=1
while (( i <= 3 )); do
  echo "step $i"
  (( i++ ))           # advance the counter - without this the loop never ends
done
```

```
step 1
step 2
step 3
```

`break` leaves a loop early and `continue` skips to the next iteration. Figure 6.1 (right) shows the loop's control flow, which `for` and `while` share: test the condition, run the body, return to the test.

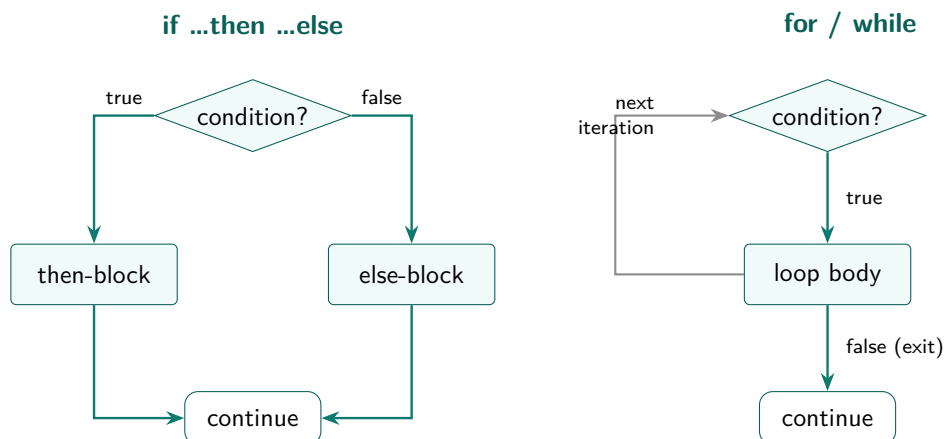


Figure 6.1: Control flow of the two core constructs. Left: a conditional chooses one branch on a test, then both paths continue. Right: a loop tests a condition, runs its body and returns to the test; `for` and `while` differ only in what the condition checks (another item in the list, or a Boolean still true).

6.7.1 Reading a file line by line

To walk over the lines of a file (a table of samples, say) use `while read`. Setting `IFS=$'\t'` splits each line on tabs into separate variables, which suits TSV files:

```
tail -n +2 expression.tsv | while IFS=$'\t' read -r gene c1 c2 c3 s1 s2 s3; do
    echo "$gene: control_1=$c1 stress_1=$s1"
done
```

```
YDL246C: control_1=142 stress_1=318
YBR118W: control_1=2011 stress_1=1877
YAL003W: control_1=1567 stress_1=1611
```

Here `read -r` disables backslash interpretation (always recommended) and `tail -n +2` skips the header line.

The pipe trap. Writing `file | while read ...` runs the loop in a **subshell**, so variables set inside it (a counter, for example) vanish when the loop ends. If the loop's result must be visible afterwards, read from a redirection instead of a pipe: `while IFS=$'\t' read -r ...; do ...; done < expression.tsv`.

6.8 Arithmetic

Integer arithmetic is done in `(( ... ))` (for control) or `$(( ... ))` (when the result is needed):

```
n=$(grep -c "^>" transcripts.fa)
echo "sequences: $n"           # sequences: 7036
echo "half: $(( n / 2 ))"      # half: 3518
```

```
sequences: 7036
half: 3518
```

`(( ... ))` works **only on integers**: `7036 / 2` gives 3518, and a division like `177 / 3` would give 59, not 59.33. For floating point, use `bc` with a set precision, or compute in `awk` with `printf "%.2f"`, often more convenient inside a script that is already working on columns:

```
echo "scale=2; 7036 / 3" | bc      # 2345.33
```

6.9 Functions

A function groups commands under a name so they can be called repeatedly. It receives arguments the same way a script does, through `$1`, `$2`, ..., and variables marked `local` exist only inside it:

```
count_seqs() {
    local file="$1"
    grep -c "^>" "$file"
}

n=$(count_seqs transcripts.fa)
echo "$n"           # 7036
```

Two things a function can “return” must be kept apart:

- its **exit status**, via **return** (a number 0 to 255 meaning success/failure, *not* a result);
- its **result**, via **echo/print**, captured with `$( ... )` at the call site (as above).

So a function that computes a value **prints** it, while **return** signals whether the operation succeeded. `local` variables protect against accidentally overwriting a variable of the same name elsewhere in the script.

6.10 Arrays

An **indexed** array holds a list of values under successive numbers:

```
groups=(control stress stress control)
echo "${groups[0]}"           # control    (first element)
echo "${#groups[@]}"          # 4           (number of elements)
echo "${groups[@]}"           # all elements
```

An **associative** array is indexed by text, letting you count “by category” directly in a script, the shell equivalent of an AWK array. It must be declared with **declare -A**. Counting the intervals per chromosome in `formats/genes.bed`:

```
declare -A per_chrom
while read -r chrom rest; do
    per_chrom[$chrom]=$(( ${per_chrom[$chrom]:-0} + 1 ))
done < formats/genes.bed
for c in "${!per_chrom[@]}"; do
    echo "$c: ${per_chrom[$c]}"
done
```

```
chrI: 2
chrII: 1
```

`${!per_chrom[@]}` gives the list of **keys**, and `${per_chrom[$c]:-0}` reads a key’s value with a default of zero when it does not yet exist.

macOS trap. Associative arrays need Bash 4 or newer. The default Bash 3.2 on macOS reports `declare: -A: invalid option`; use the `conda` environment with a modern Bash (the counts above were verified against the data).

6.11 Putting it together

The constructs combine into a small, self-contained tool. The script below summarises a BED file. It validates its argument, then reports the number of intervals and the total length covered, computing the length in `awk`:

```
#!/usr/bin/env bash
# bedinfo.sh - summarise a BED file: number of intervals and total length

if (( $# == 0 )); then
    echo "usage: $0 <file.bed>" >&2
    exit 1
fi

bed="$1"
if [[ ! -s "$bed" ]]; then
    echo "error: '$bed' is missing or empty" >&2
    exit 1
fi

name="${bed##*/}"
n=$(grep -c . "$bed")
total=$(awk '{s += $3 - $2} END{print s}' "$bed")

echo "file:      $name"
echo "intervals: $n"
echo "total bp:  $total"
```

Running it on the example file, and then with no argument:

```
./bedinfo.sh formats/genes.bed
```

```
file:      genes.bed
intervals: 3
total bp:  4003
```

```
./bedinfo.sh
```

```
usage: ./bedinfo.sh <file.bed>
```

The total, 4003, is the sum of the three interval lengths, the same BED arithmetic (`end - start`) seen in Chapter 4. Error messages go to standard error with `>&2`, so they stay separate from the script's real output and are not captured by a `$( ... )` further down a pipeline.

6.12 Common mistakes

Worth checking before handing in a script:

1. **Spaces around =.** `x = 5` is an error; assignment is `x=5`, no spaces.
2. **Unquoted variables.** `$file` without quotes breaks on spaces and wildcards. Default to `"$file"`.
3. **Subshell in ... | while read.** Variables set inside the loop are lost after the pipe; read from a redirection `< file` instead.
4. **Mixing comparison operators.** `-eq/-lt` are for numbers, `=/<` for strings; swapping them fails silently (`"10" < "9"` is true as text, false as numbers).
5. **Globs with no match.** `for f in *.fasta` in an empty directory substitutes the literal `*.fasta`; test `[[ -f "$f" ]]` inside the loop.
6. **Running without ./.** `script.sh` searches the `PATH`; your own script runs as `./script.sh` (and needs `chmod +x`).
7. **Windows \r line endings (CRLF).** A script saved by a Windows editor may fail with bad interpreter or command not found. Fix with `sed -i 's/\r$//' script.sh`.
8. **Associative arrays on Bash 3.2 (macOS).** They need Bash 4+ from the conda environment.

6.13 Command summary

Form	Effect
<code>#!/usr/bin/env bash</code>	shebang, selects the interpreter (portable)
<code>chmod +x script.sh</code>	make a script executable
<code>./script.sh</code>	run a script via its shebang
<code>x="value"</code>	assign a variable (no spaces around =)
<code>"\$x"</code>	use a variable (quoted)
<code>\$(command)</code>	capture a command's output
<code>\$1, \$@, \$#, \$0</code>	arguments; all; count; script name
<code>\${f##*/}</code>	bare file name (strip directory)
<code>\${f%.bed}</code>	name without extension
<code>\${v:-default}</code>	value, or default if empty
<code>if ...; then ...; fi</code>	conditional
<code>[[-f "\$p"]]</code>	test: <code>p</code> is a file (! negates)
<code>[["\$a" = "\$b"]]</code>	string comparison; numbers: <code>-eq/-lt/-gt</code>
<code>case "\$x" in pat) ... ;; *) ... ;; esac</code>	choose a branch by pattern
<code>exit N</code>	end the script with status <code>N</code>
<code>for f in *.bed; do ...; done</code>	loop over files
<code>while ((cond)); do ...; done</code>	loop while a condition holds
<code>while IFS=\$'\t' read -r a b; do ...; done < file</code>	read a TSV line by line
<code>((a + b)), \$((a / b))</code>	integer arithmetic
<code>echo "scale=2; a/b" \ bc</code>	floating-point arithmetic
<code>name() { ...; }</code>	define a function
<code>local v=...</code>	function-local variable
<code>arr=(a b c), \${arr[@]}, \${#arr[@]}</code>	indexed array
<code>declare -A m; m[key]=x</code>	associative array (Bash 4+)

6.14 Materials

- **ShellCheck**, <https://www.shellcheck.net/>, paste in a script and it flags bugs and risky constructs (missing quotes, wrong comparisons). Introduced properly in Chapter 7, but useful from the start.
- **explainshell**, <https://explainshell.com/>, breaks any command line into its documented options.
- **Bash Guide** (<https://mywiki.woledge.org/BashGuide>) and **BashPitfalls** (<https://mywiki.woledge.org/BashPitfalls>), a reliable practical guide and a catalogue of common errors.
- **GNU Bash Reference Manual**, <https://www.gnu.org/software/bash/manual/>, the complete reference for a specific construct.
- **Google Shell Style Guide**, <https://google.github.io/styleguide/shellguide.html>, concise rules for readable scripts.

6.15 Exercises

The exercises use the course data package; copy the files you need into a working directory first.

Exercise 6.1. Write a script `seqcount.sh` that takes one FASTA file as an argument and prints how many sequences it contains (count of `>` headers). If no argument is given, print a usage message to standard error and exit with status 1. Test it on `transcripts.fa`.

Exercise 6.2. Extend `seqcount.sh` to accept *several* FASTA files and report the count for each, using a `for` loop over `"$@"` and parameter expansion to print the bare file name. Test it on several copies of a FASTA file with different names.

Exercise 6.3. Using parameter expansion only (no `sed/cut`), turn the path `data/samples/control_1.fastq.gz` into (a) the bare file name, (b) the file name without any extension, and (c) the containing directory. Verify each against `basename/dirname`.

Exercise 6.4. Write a loop over `formats/*.bed` that, for each file, reports whether it is empty or how many intervals it has, using an `if [[ -s ... ]]` test. Add a `case` on the file name that labels each file by type.

Exercise 6.5. Read `expression.tsv` with `while IFS=$'\t' read -r`, skipping the header, and print each gene whose `control_1` value exceeds the mean of all `control_1` values. (Compute the mean first with `awk`, store it in a variable, and compare numerically.)

Exercise 6.6. Write a function `gc_content` that takes a DNA string and prints its GC percentage to one decimal place (use `tr -cd` to count bases, or `awk`). Call it on a few short sequences and check the result by hand for one of them.

Exercise 6.7. Turn `bedinfo.sh` from this chapter into a script that accepts *many* BED files and prints a one-line summary per file (name, interval count, total bp), skipping any argument that is missing or empty with a warning to standard error.

Chapter 7

Implementing software in the Linux environment

The previous chapter showed how to put commands into a script so they can be rerun and reused. A script that only you can run, only on your machine, only today, is already useful, but the step from a *working script* to a *tool* is what makes an analysis trustworthy. A tool announces how it should be called, refuses bad input instead of producing nonsense, stops the moment something goes wrong rather than pressing on with half-written files, cleans up after itself, and can be read, checked and reproduced by someone else (including yourself in six months). This chapter is about that step: the handful of techniques that turn a shell script into a small, robust program, and the three tools (Git, Conda and Docker) that let an analysis be reproduced on another computer.

Why this matters. A result nobody can reproduce (not a reviewer, not a collaborator, not its own author a year later) has little scientific value. Reproducibility is a requirement, not a decoration, and it is built in layers: a tool that behaves predictably and fails loudly (this is what `getopts`, `set -euo pipefail` and `trap` give you), a record of *what changed and why* (Git), a fixed set of program versions (Conda), and, when needed, the whole system packaged together (Docker). This chapter builds those layers from the inside out.

Working on macOS or Windows? The strict-mode and `trap` examples below behave the same everywhere. As in the previous chapter, the course uses a `conda` environment providing a modern Bash so that associative arrays and other recent features work identically on macOS (whose system Bash is the old 3.2) and on Windows under WSL. `conda` and `docker` are installed separately; the commands shown for them follow the official documentation, and you run them in your own environment.

7.1 From a script to a tool: options with `getopts`

A real tool is called with **named options**, not a fixed order of arguments: `-i input.fa, -m 1000, -h` for help. Named options are self-documenting, can appear in any order, and have sensible defaults. Bash provides `getopts` to parse them. The pattern is always the same: a `while` loop around `getopts`, a `case` that handles each letter, and the special variable `OPTARG` holding the value of an option that takes one:

```
while getopts "i:m:h" opt; do
    case "$opt" in
        i) infile="$OPTARG" ;;      # -i takes a value (the colon after i)
        m) minlen="$OPTARG" ;;      # -m takes a value
        h) usage; exit 0 ;;          # -h is a flag (no colon, no value)
```

```

        *) usage; exit 1 ;;          # anything else: show usage, fail
    esac
done

```

The string "i:m:h" is the **option specification**: a letter followed by a colon takes an argument (i:, m:), a bare letter is a flag (h). Inside the loop, `opt` holds the current option letter and `OPTARG` holds its value. An unrecognised option falls through to `*`. This loop is the front door of almost every command-line tool you will write.

7.2 Validating input and failing loudly

A tool that is given bad input should say so clearly and stop, not crash halfway or silently produce an empty result. Two habits make this reliable: send error messages to **standard error** (`>&2`, so they are not mistaken for output and are not captured by a pipe), and return a **non-zero exit status** so that whatever called the tool knows it failed. A `usage()` function printed with a here-document keeps the help text in one place:

```

usage() {
    cat >&2 <<USAGE
Usage: fainfo.sh -i FILE [-m MINLEN] [-h]
    -i FILE      input FASTA file (required)
    -m MINLEN    report only sequences of at least MINLEN bp (default 0)
    -h          show this help and exit
USAGE
}

if [[ -z "$infile" ]]; then
    echo "fainfo.sh: an input file is required (-i)" >&2
    usage
    exit 1
fi
if [[ ! -s "$infile" ]]; then
    echo "fainfo.sh: '$infile' is missing or empty" >&2
    exit 1
fi

```

The test `[[ -z "$infile" ]]` is true when the variable is empty (no `-i` was given); `[[ ! -s "$infile" ]]` is true when the file does not exist or has zero size. Each failure prints a specific message to standard error and exits with status 1. By convention 0 means success and any non-zero value means failure; a tool may use different numbers for different failures so that scripts calling it can react.

7.3 Running safely: set -euo pipefail

By default, Bash keeps going after a command fails. If an early step produces no output, the rest of the script happily works on nothing and may overwrite a good file with an empty one. Three options, set together at the top of every serious script, change this:

```
set -euo pipefail
```

Option	Effect
<code>set -e</code>	stop the script as soon as any command exits non-zero
<code>set -u</code>	treat the use of an unset variable as an error (catches typos in names)
<code>set -o pipefail</code>	a pipeline fails if <i>any</i> stage fails, not only the last

The `pipefail` part fixes a subtle trap. The exit status of a pipeline is normally the status of its **last** command, so a failure earlier in the pipe is hidden:

```
cat no_such_file.fa 2>/dev/null | head -1 > /dev/null; echo "exit=$?"
```

```
exit=0
```

The `cat` failed, but `head` succeeded, so the pipeline reports success. With `pipefail` the earliest failure surfaces:

```
set -o pipefail
cat no_such_file.fa 2>/dev/null | head -1 > /dev/null; echo "exit=$?"
```

```
exit=1
```

Figure 7.1 shows how the exit status drives both `set -e` and `pipefail`.

One consequence of `set -e` deserves a mention. Some commands return a non-zero status as *information*, not as an error: `grep` exits 1 when it simply finds no match. Under `set -e` that would abort the script:

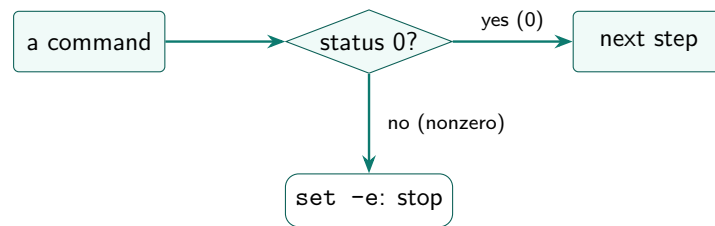
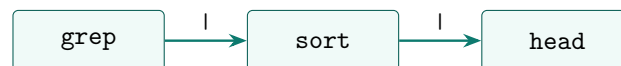
```
set -e
n=$(grep -c "^@ZZZ" formats/reads.fastq); echo "reached: n=$n"
```

(the script stops here: `grep` found nothing, exited 1, `set -e` aborted)

Guard such commands with `|| true` so a “no match” counts as success:

```
set -e
n=$(grep -c "^@ZZZ" formats/reads.fastq || true); echo "reached: n=$n"
```

```
reached: n=0
```

exit status**pipeline status**

default: \$? is the *last* stage (head)

set -o pipefail: \$? is the *first* failing stage

Figure 7.1: How the exit status controls strict mode. Top: under `set -e`, a command whose status is not zero stops the whole script instead of letting later steps run on bad data. Bottom: the status of a pipeline is, by default, that of its last stage, so an early failure is hidden; `set -o pipefail` makes the first failing stage decide the result.

7.4 Cleaning up: trap and mktemp

Tools often need scratch files. Two problems follow: choosing a name that will not clash with another process, and making sure the file is removed even if the script exits early. `mktemp` solves the first by creating a uniquely named temporary file and printing its path; `trap` solves the second by registering a command to run when the script exits, *however* it exits:

```
#!/usr/bin/env bash
set -euo pipefail

tmp="$(mktemp)"
trap 'rm -f "$tmp"' EXIT          # remove the temp file on any exit

grep ">" transcripts.fa > "$tmp"
echo "the file has $(wc -l < "$tmp" | tr -d ' ') sequence headers"
```

the file has 7036 sequence headers

The `trap '...' EXIT` line means “run `rm -f "$tmp"` when the shell exits”, on normal completion, on an error under `set -e`, or on an interrupt. The temporary file is therefore cleaned up without a `rm` at the end of every possible path through the script. This is the standard idiom for safe scratch space.

7.5 Finding mistakes: `bash -n`, `set -x` and `shellcheck`

Three tools shorten the distance from “it does not work” to “here is the line”. `bash -n` parses a script **without running it** and reports syntax errors, a cheap check to run before every execution:

```
bash -n myscript.sh
```

```
myscript.sh: line 3: syntax error near unexpected token `echo'
myscript.sh: line 3: `echo "$f"'
```

`set -x` turns on a **trace**: Bash prints each command, with variables expanded, just before running it, invaluable for seeing what a loop actually did. Switch it on around the suspect section and off again with `set +x`:

```
set -x
for f in formats/*.bed; do wc -l "$f"; done
set +x
```

The third tool, **ShellCheck**, is a static analyser written specifically for shell scripts. It reads a script and reports likely bugs, each with a stable code you can look up. It is not installed by default; you run it on your own machine (it is available through `conda`, see *Materials*). Common findings, straight from its documentation, include:

Code	What it warns about
SC2086	an unquoted variable (<code>\$f</code> instead of <code>"\$f"</code>) that will break on spaces
SC2046	unquoted command substitution that word-splits unexpectedly
SC2164	<code>cd somewhere</code> without <code> exit</code> , which keeps going in the wrong place
SC2148	a missing shebang line

Running `shellcheck myscript.sh` prints each issue with its line number, the code, and a short explanation; fixing the warnings it raises eliminates most of the quoting and portability bugs that bite shell scripts.

7.6 Combining programs: process substitution

Some commands compare two inputs that each come from a command, not a file: `diff`, `comm`, `join`, `paste`. **Process substitution**, written `<(...)`, runs a command and presents its output *as if it were a file*, so these tools can read it directly without a temporary file. To find the genes common to two unsorted lists:

```
comm -12 <(sort genes_set_a.txt) <(sort genes_set_b.txt) | wc -l
```

20

`comm` requires both inputs sorted and compares them line by line: `-12` keeps only the lines present in *both* (suppressing columns 1 and 2, the unique-to-each columns). Here 20 of the 40 genes are shared. The same technique compares the output of two pipelines directly:

```
diff <(cut -f1 run1.tsv | sort) <(cut -f1 run2.tsv | sort)
```

Without process substitution each side would need its own temporary file; `<(...)` keeps the whole comparison on one line.

7.7 Joining tables by a shared key: `join`

Biological analysis constantly needs to combine tables on a shared identifier: attach gene names to expression values, merge counts with annotation. `join` does this the way a database does, matching rows by a key column. Its one firm requirement is that **both inputs are sorted on the join key**.

Suppose `expression.tsv` holds counts keyed by gene ID, and a second table maps each gene ID to its name (built here with one `awk` line over the annotation, then sorted):

```
# gene_id -> gene_name, sorted on the key
awk -F'\t' ' $3=="gene"{
    g=$9; sub(/.*gene_id "/, "", g); sub(/".*/, "", g)
    n=$9; if (n ~ /gene_name/){ sub(/.*gene_name "/, "", n); sub(/".*/, "", n) }
    else n="."
    print g"\t"n
}' annotation.gtf | sort -k1,1 > names.tsv

# sort the expression table on its key too, dropping the header
tail -n +2 expression.tsv | sort -k1,1 > expr_sorted.tsv
```

Now `join` matches the two on their first column. The `-o` option chooses which fields to print, here the gene ID, its name from the second file, and the first count from the first file:

```
join -t$'\t' -1 1 -2 1 -o '1.1,2.2,1.2' expr_sorted.tsv names.tsv
```

YAL003W	EFB1	1567
YBR118W	TEF2	2011
YDL246C	SOR2	142
YDR155C	CPR1	998
YER165W	PAB1	45
YFL039C	ACT1	2890
YGL008C	PMA1	88
YGR192C	TDH3	4102
YHR174W	ENO2	3450
YKL060C	FBA1	1203
YLR044C	PDC1	560
YMR186W	HSC82	1340

```
YNL209W SSB2      720
YOR369C RPS12     310
YPL240C HSP82     1780
```

`-t$'\t'` sets a tab as the field separator; `-1 1` and `-2 1` name the key column in each file; `-o '1.1,2.2,1.2'` means “field 1 of file 1, field 2 of file 2, field 2 of file 1”. All 15 genes matched, now carrying their familiar names (YFL039C is ACT1, YGR192C is TDH3). If the output is empty or short, the usual cause is that the inputs were not sorted on the key, the single most common join mistake.

7.8 Doing work in parallel: xargs and GNU parallel

When the same independent command must run on many files, running them one at a time wastes the other cores of the machine. **xargs** `-P N` runs up to `N` jobs at once, feeding each a file name. The `-I{}` option puts the file name wherever `{}` appears:

```
ls *.bed *.vcf *.sam *.fastq | xargs -P 4 -I{} \
    bash -c 'printf "%-16s %s\n" "{}" "$(wc -l < "{}" | tr -d " ")"'
```

```
alignment.sam      8
reads.fastq        12
genes.bed           3
peaks.bed           4
variants.vcf       11
```

Four jobs run concurrently, so the output order is not necessarily the input order, a sign the work really was parallel. **GNU parallel** is a richer alternative with a clearer syntax, and it is designed for exactly this task; it is installed separately (see *Materials*), and you run it in your own environment:

```
parallel -j 4 'wc -l {}' ::: *.bed *.vcf *.sam *.fastq
```

parallel adds conveniences such as `{.}` (the file name without its extension) and `{/}` (the bare name), automatic grouping of each job’s output, and an easy way to build argument combinations. For a few files **xargs** `-P` is enough; for a real batch of samples **parallel** is worth learning.

7.9 Version control with Git

A tool is also a document, and documents change. **Git** records the history of a project as a sequence of snapshots (*commits*), each with a message saying what changed and why, so that any earlier state can be recovered and every change is accounted for. It is distributed (the full history lives in the project directory, in a hidden `.git` folder), and it works entirely offline; a remote such as GitHub is a copy for sharing and backup.

Files move through three areas: the **working directory** (what you edit), the **staging area** (changes marked for the next snapshot, with `git add`), and the **history** (committed snapshots, with `git commit`). A first commit looks like this (the commands were run for real; your commit identifier will differ, because Git computes it from the content, author and time):

```
git init
git config user.name "Your Name"
git config user.email "you@example.org"
printf '%s\n' '#!/usr/bin/env bash' 'grep -c "^>" "$1"' > countfa.sh
printf '%s\n' 'results/' '*.bam' > .gitignore
git add countfa.sh .gitignore
git commit -m "Add countfa.sh and gitignore"
git log --oneline
```

```
8bba8d0 Add countfa.sh and gitignore
```

After editing a tracked file, `git diff` shows what changed, and `git diff --stat` summarises it:

```
echo '# counts FASTA records' >> countfa.sh
git diff --stat
```

```
countfa.sh | 1 +
1 file changed, 1 insertion(+)
```

The `.gitignore` file lists paths Git should not track (results and large data), which can be regenerated from code and would bloat the repository. With `results/` and `*.bam` ignored, a freshly created `results/out.bam` does not even appear as an untracked file:

```
mkdir -p results && touch results/out.bam
git status -s
```

```
M countfa.sh
```

Only the modified script is listed; the ignored output is invisible to Git. The rule of thumb for a bioinformatics repository: **version the code and configuration that produce the results, not the results themselves** (and never commit passwords or API keys). A branch (`git checkout -b feature`) lets an experiment proceed without touching the stable `main`, and `git merge` brings it back when it works; `git push` and `git pull` exchange history with a remote.

7.10 Reproducible environments: Conda and Docker

Git versions the code, but not the *programs* the code calls. An analysis run with `samtools` 1.9 may give different results under 1.19. **Conda** manages this: an **environment** is an isolated set of packages at fixed versions, described in a file that is versioned alongside the code. The description lives in `environment.yml`:

```
name: codinglab
channels:
  - conda-forge
  - bioconda
```

```
dependencies:
- python=3.11
- samtools=1.19
- bcftools=1.19
```

and the whole environment is recreated with one command (the commands follow the official Conda and Bioconda documentation; the installation result is produced in your own environment):

```
conda env create -f environment.yml
conda activate codinglab
```

Channel order matters: **conda-forge** must sit above **bioconda**, because Bioconda builds on it. One **git clone** plus one **conda env create** then reproduce both the code and the versions it needs.

The widest layer is **Docker**, which packages the whole system, operating system, libraries and tools, into an **image** that runs identically anywhere. Writing a **Dockerfile** is beyond this course, and most of the time it is unnecessary: the **BioContainers** project publishes ready-made Docker images for thousands of bioinformatics tools, built automatically from Bioconda, so the exact same version of a tool can be run with **docker run** on any machine that has Docker. Figure 7.2 places the three tools in order of scope.

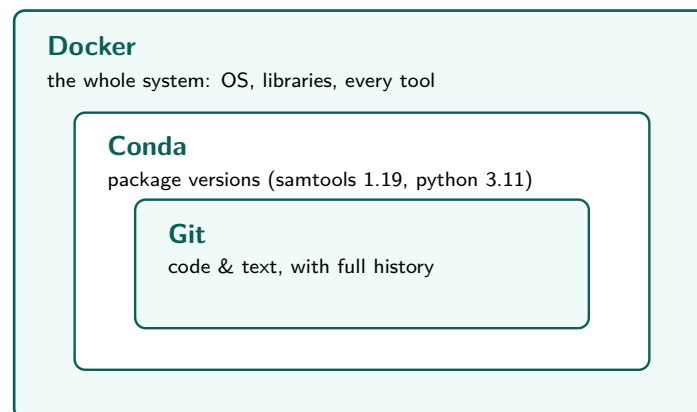


Figure 7.2: Three layers of reproducibility, from the narrowest scope to the widest. *Git* records which code produced a result; *Conda* fixes the versions of the programs that code runs; *Docker* captures the entire system around them. For day-to-day work *Git* and *Conda* are usually enough; *Docker* (or a ready-made *BioContainers* image) is added when an analysis must move to another machine or into a publication.

7.11 Putting it together: a complete tool

The pieces above combine into a small but genuine tool. **fainfo.sh** parses options with **getopts**, validates its input and fails loudly, runs under strict mode, and does its real work with **awk**:

```
#!/usr/bin/env bash
set -euo pipefail

usage() {
```

```

    cat >&2 <<USAGE
Usage: fainfo.sh -i FILE [-m MINLEN] [-h]
    -i FILE      input FASTA file (required)
    -m MINLEN    report only sequences of at least MINLEN bp (default 0)
    -h           show this help and exit
USAGE
}

infile=""
minlen=0
while getopts "i:m:h" opt; do
    case "$opt" in
        i) infile="$OPTARG" ;;
        m) minlen="$OPTARG" ;;
        h) usage; exit 0 ;;
        *) usage; exit 1 ;;
    esac
done

if [[ -z "$infile" ]]; then
    echo "fainfo.sh: an input file is required (-i)" >&2
    usage
    exit 1
fi
if [[ ! -s "$infile" ]]; then
    echo "fainfo.sh: '$infile' is missing or empty" >&2
    exit 1
fi

awk -v min="$minlen" '
    /^>/ { if (name != "" && len >= min) { n++; tot += len }
           name = $0; len = 0; next }
    { len += length($0) }
    END { if (name != "" && len >= min) { n++; tot += len }
          printf "sequences: %d\n", n
          if (n > 0) printf "mean length: %.1f bp\n", tot / n }
' "$infile"

```

Run with no arguments, it refuses and explains itself, returning a non-zero status:

```
./fainfo.sh; echo "exit=$?"
```

```

fainfo.sh: an input file is required (-i)
Usage: fainfo.sh -i FILE [-m MINLEN] [-h]
    -i FILE      input FASTA file (required)
    -m MINLEN    report only sequences of at least MINLEN bp (default 0)
    -h           show this help and exit
exit=1

```

Given a file, it reports; given a threshold, it filters:

```
./fainfo.sh -i transcripts.fa
```

```
sequences: 7036
mean length: 1258.8 bp
```

```
./fainfo.sh -i transcripts.fa -m 1000
```

```
sequences: 3506
mean length: 2036.2 bp
```

The options can appear in any order, the defaults are sensible, bad input is rejected with a clear message, and a failure is reported through the exit status, the difference between a script and a tool.

7.12 Common mistakes

1. **No strict mode.** Without `set -euo pipefail`, a failing step is ignored and the script runs on bad data. Put it at the top of every serious script.
2. **Forgetting that `grep` exits 1 on no match.** Under `set -e` this aborts the script. Guard counts and searches that may legitimately find nothing with `|| true`.
3. **A pipeline that hides failures.** Without `pipefail`, only the last stage's status counts, so `badcommand | sort` looks successful. Enable `pipefail`.
4. **Unquoted variables.** `$f` breaks on file names with spaces; always write `"$f"`. ShellCheck (SC2086) flags every instance.
5. **Join on unsorted input.** `join` silently returns too few rows when its inputs are not sorted on the key. Sort both files on the join column first.
6. **Leaving scratch files behind.** A `rm` at the end of the script is skipped when the script exits early. Use `trap 'rm -f "$tmp"' EXIT` with `mktemp` instead.
7. **Committing results and large data to Git.** The repository balloons and clones crawl. List results and `*.bam/*.vcf.gz` in `.gitignore`; version code, not data.
8. **Unpinned package versions.** `dependencies: [samtools]` installs whatever is newest, which changes over time. Pin versions (`samtools=1.19`) in `environment.yml`.

7.13 Command summary

Task	Command
parse named options	<code>while getopts "i:m:h" opt; do case "\$opt" in ...; esac; done</code>
value of an option	<code>"\$OPTARG"</code>
strict mode	<code>set -euo pipefail</code>
allow “no match” under <code>set -e</code>	<code>cmd true</code>
message to standard error	<code>echo "..." >&2</code>
exit with a status	<code>exit 1</code> (0 = success, non-zero = failure)
unique temp file	<code>tmp="\$(mktemp)"</code>

Task	Command
clean up on exit	<code>trap 'rm -f "\$tmp"' EXIT</code>
syntax-check, no run	<code>bash -n script.sh</code>
trace execution	<code>set -x ... set +x</code>
static analysis	<code>shellcheck script.sh</code>
command output as a file	<code><(command)</code>
compare two command outputs	<code>diff <(...) <(...) / comm -12 <(...) <(...)</code>
join two tables on a key	<code>join -t\$'\t' -1 1 -2 1 file1 file2</code> (both sorted)
run N jobs in parallel	<code>ls ... \ xargs -P N -I{} cmd {} /</code> <code>parallel -j N 'cmd {}' ::: ...</code>
start a repository	<code>git init</code>
stage / commit	<code>git add <file> / git commit -m "..."</code>
history / changes	<code>git log --oneline / git diff / git</code> <code>status -s</code>
recreate an environment	<code>conda env create -f environment.yml</code>

7.14 Materials

All resources below are open-access.

- **GNU Bash manual** (shell options, `getopts`, `trap`), <https://www.gnu.org/software/bash/manual/>
- **ShellCheck**, the static analyser, with an explanation page for every code, <https://www.shellcheck.net/>
- **explainshell**, pastes any command line and labels each part, <https://explainshell.com/>
- **GNU parallel documentation**, <https://www.gnu.org/software/parallel/>
- **Pro Git** (Scott Chacon, Ben Straub), the complete Git book, free online, <https://git-scm.com/book>
- **Conda, managing environments**, <https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html>
- **Bioconda**, channel setup and package index, <https://bioconda.github.io/>
- **BioContainers**, ready-made container images for bioinformatics tools, <https://biocontainers.pro/>
- **The Turing Way**, an open handbook of reproducible research, <https://the-turing-way.netlify.app/>
- **Google Shell Style Guide**, <https://google.github.io/styleguide/shellguide.html>

7.15 Exercises

The exercises use the course data package; copy the files you need into a working directory first. Exercises 7.1 to 7.5 cover the core techniques of this chapter and are enough to show you have understood it. Exercises 7.6 and 7.7 are marked *going further*: they are optional and a little more demanding, and are best attempted once the earlier ones feel comfortable.

Exercise 7.1. Take `seqcount.sh` from the previous chapter and rewrite its interface with `getopts`: accept `-i FILE` (required) and `-h` (help). If `-i` is missing, print a usage message to standard error and exit with status 1. Confirm that `-h` prints the help and exits 0, and that the options work in any order.

Exercise 7.2. Add `set -euo pipefail` to the top of your script from 7.1 and a validation check that rejects a missing or empty input file with a clear message. Then deliberately introduce a `grep -c` on a pattern that matches nothing and show that the script now aborts; fix it with `|| true` and explain why.

Exercise 7.3. Write a script that creates a temporary file with `mktemp`, writes the sequence headers of a FASTA file into it, reports how many there are, and removes the temporary file with a `trap ... EXIT`. Verify that the temporary file is gone after the script finishes.

Exercise 7.4. Using process substitution, report how many gene identifiers are **unique to** `genes_set_a.txt` (present in A but not in B) with a single `comm` command, without creating any intermediate files. Then do the same for set B.

Exercise 7.5. Build a sorted `gene_id-to-gene_name` table from `annotation.gtf` and join it to `expression.tsv` so that each row carries the gene name alongside its counts. Deliberately leave one input unsorted and describe what happens to the output.

Exercise 7.6 (going further, optional). Use `xargs -P` (or `parallel`, if installed) to count the records in every file under `formats/` concurrently, printing one line per file. Explain why the order of the output lines may change between runs.

Exercise 7.7 (going further, optional). Create a small Git repository containing your tool from 7.1 and an `environment.yml` that pins Bash and `awk`'s provider. Add a `.gitignore` that excludes a `results/` directory, make two commits with meaningful messages, and show the history with `git log --oneline`. (Your commit identifiers will differ from anyone else's, explain why.)

Chapter 8

Writing custom scripts for biological data processing

Every earlier chapter added one instrument to the toolbox: the command line and its file-handling commands, the small tools that filter and reshape text, the VIM editor, the file formats that carry biological data, AWK for columnar computation, Bash for control logic, and the techniques that turn a script into a robust, reproducible tool. This final chapter puts them to work together. Real analysis is rarely a single command; it is a **pipeline** (a sequence of stages, each reading what the previous one produced) that carries raw data through to a result you can act on. Here we build one such pipeline end to end, on the course data, and wrap it in a single script that anyone can run.

Why a capstone. Learning each tool in isolation answers the question “what does this command do?”. Assembling them answers the one that matters in practice: “how do I get from the files I was given to the answer I need?”. The pipeline below is small (a few dozen lines) but it is complete: it reads two real files, computes something biologically meaningful, annotates and classifies the result, validates its input, cleans up after itself, and prints a report. Scaling this pattern up is what day-to-day bioinformatics programming looks like.

8.1 The shape of an analysis

The question we will answer is simple to state: **which genes respond to stress?** The course package provides what we need: `expression.tsv`, a table of read counts for a set of genes under three control and three stress replicates, and `annotation.gtf`, from which we can recover each gene’s name. The analysis has a natural shape: two independent branches that each prepare one input, a step that merges them on the shared gene identifier, and a final step that classifies and sorts the result. Figure 8.1 draws it as a directed graph, the standard way to picture a pipeline, where each box is a stage and each arrow is data flowing from one to the next.

8.2 Branch A: gene names from the annotation

The expression table identifies genes by their systematic ID (YDL246C), but a report is far more readable with the common name (SOR2). Those names live in the ninth column of the GTF, inside the attribute string. One `awk` program pulls out the `gene_id` and `gene_name` of every `gene` feature and prints them as a two-column table, sorted on the ID so it is ready to join:

```
awk -F'\t' ' $3=="gene"{
    g=$9; sub(/.*gene_id "/, "", g); sub(/".*/, "", g)
    n=$9; if (n ~ /gene_name/){ sub(/.*gene_name "/, "", n); sub(/".*/, "", n) }
    else n="."
}
```

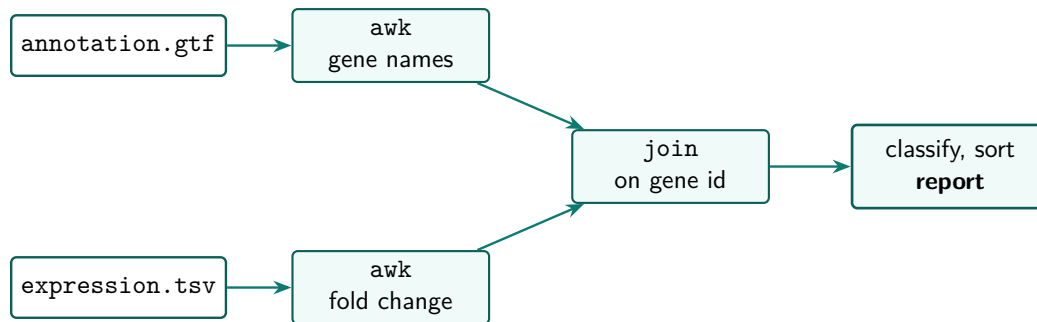


Figure 8.1: The analysis as a directed graph. Two branches prepare the two inputs independently: one extracts gene names from the annotation, the other computes a stress/control fold change from the counts. A *join* merges them on the shared gene identifier, and a final stage classifies each gene and sorts the table into a report. Each box is one stage; the arrows are files passing from stage to stage.

```

print g"\t"n
}' annotation.gtf | sort -k1,1 > names.tsv

```

Each `sub()` trims the attribute string down to the quoted value: the first removes everything up to the opening quote, the second removes the closing quote and whatever follows. Genes without a `gene_name` attribute get a `.` placeholder, so every gene still produces a row.

8.3 Branch B: fold change from the counts

The second branch reduces the six count columns to a single number per gene. For each row, it averages the three control replicates and the three stress replicates, then divides to get the **fold change**, how many times higher (or lower) expression is under stress. The header line is dropped first with `tail -n +2`:

```

tail -n +2 expression.tsv | awk -F'\t' 'BEGIN{OFS="\t"}{
    c = ($2+$3+$4)/3          # mean of the three control replicates
    s = ($5+$6+$7)/3          # mean of the three stress replicates
    printf "%s\t%.1f\t%.1f\t%.2f\n", $1, c, s, s/c
}' | sort -k1,1 > fc.tsv

```

```

YAL003W 1555.7 1630.0 1.05
YBR118W 2034.0 1867.0 0.92
YDL246C 143.7 358.7 2.50
...
YER165W 48.3 396.0 8.19

```

A fold change near 1.00 means stress barely moved the gene; 8.19 (for YER165W) means its expression rose roughly eight-fold.

8.4 Merging the branches: join, classify, sort

Now the two branches meet. `join` matches `fc.tsv` and `names.tsv` on the gene ID and emits the name alongside the numbers; a second `awk` labels each gene **induced**, **repressed** or **unchanged**

by comparing its fold change against a threshold; and `sort` orders the table by fold change, largest first:

```
join -t$'\t' -1 1 -2 1 -o '1.1,2.2,1.2,1.3,1.4' fc.tsv names.tsv \
| awk -F'\t' 'BEGIN{OFS="\t"}{
    cls = ($5>=2 ? "induced" : ($5<=0.5 ? "repressed" : "unchanged"))
    print $0, cls
}' \
| sort -t$'\t' -k5,5nr
```

Each stage does one job and hands its output to the next, exactly the composition of small tools that the whole course has been building toward. The result is a table of genes ranked by how strongly they responded to stress, each with its familiar name.

8.5 From commands to a tool

Typed at the prompt, this works once. Saved as a script with a proper interface, it becomes a tool: it takes its two inputs as named options, lets the user choose the threshold, validates everything, runs under strict mode, and cleans up its scratch files with `trap`. This is the whole analysis as one program, `expression_report.sh`:

```
#!/usr/bin/env bash
# expression_report.sh - classify genes by stress/control fold change,
# annotate them with gene names, and print a sorted report.
set -euo pipefail

usage() {
    cat >&2 <<USAGE
Usage: expression_report.sh -e EXPR.tsv -a ANNOT.gtf [-f FOLD] [-h]
    -e EXPR      expression table: gene + 3 control + 3 stress columns (required)
    -a ANNOT      GTF annotation, for gene names (required)
    -f FOLD      induction threshold (default 2.0); repression is its reciprocal
    -h          show this help and exit
USAGE
}

expr="" annot="" fold=2.0
while getopts "e:a:f:h" opt; do
    case "$opt" in
        e) expr="$OPTARG" ;;
        a) annot="$OPTARG" ;;
        f) fold="$OPTARG" ;;
        h) usage; exit 0 ;;
        *) usage; exit 1 ;;
    esac
done

[[ -z "$expr" ]] && { echo "error: -e is required" >&2; usage; exit 1; }
[[ -z "$annot" ]] && { echo "error: -a is required" >&2; usage; exit 1; }
[[ ! -s "$expr" ]] && { echo "error: '$expr' missing or empty" >&2; exit 1; }
[[ ! -s "$annot" ]] && { echo "error: '$annot' missing or empty" >&2; exit 1; }
```

```

tmp="$(mktemp -d)"
trap 'rm -rf "$tmp"' EXIT

# branch A: gene_id -> gene_name
awk -F'\t' ' $3=="gene"{
    g=$9; sub(/.*gene_id "/", "", g); sub(/".*/", "", g)
    n=$9; if (n ~ /gene_name/){ sub(/.*gene_name "/", "", n); sub(/".*/", "", n) }
    else n="."
    print g"\t"n
}' "$annot" | sort -k1,1 > "$tmp/names.tsv"

# branch B: fold change per gene
tail -n +2 "$expr" | awk -F'\t' 'BEGIN{OFS="\t"}{
    c=($2+$3+$4)/3; s=($5+$6+$7)/3; printf "%s\t%.1f\t%.1f\t%.2f\n", $1, c, s, s/c
}' | sort -k1,1 > "$tmp/fc.tsv"

# merge, classify, sort, print
join -t$'\t' -1 1 -2 1 -o '1.1,2.2,1.2,1.3,1.4' "$tmp/fc.tsv" "$tmp/names.tsv" \
    | awk -F'\t' -v up="$fold" 'BEGIN{OFS="\t"; dn=1/up}{
    cls=($5>=up?"induced":($5<=dn?"repressed":"unchanged")); print $0,cls
}' \
    | sort -t$'\t' -k5,5nr \
    | awk 'BEGIN{printf "%-9s %-7s %9s %9s %7s\n", "gene", "name", "control", "stress", "fold", "class"}
    {printf "%-9s %-7s %9s %9s %7s\n", $1, $2, $3, $4, $5, $6; n[$6]++}
    END{print ""; for(c in n) printf " %s: %d\n", c, n[c] | "sort"}'

```

Called with no arguments, it explains itself and fails:

```
./expression_report.sh
```

```
error: -e is required
```

```
Usage: expression_report.sh -e EXPR.tsv -a ANNOT.gtf [-f FOLD] [-h]
```

```

-e EXPR      expression table: gene + 3 control + 3 stress columns (required)
-a ANNOT     GTF annotation, for gene names (required)
-f FOLD      induction threshold (default 2.0); repression is its reciprocal
-h           show this help and exit

```

Given its two inputs, it produces the finished report:

```
./expression_report.sh -e expression.tsv -a annotation.gtf
```

gene	name	control	stress	fold	class
YER165W	PAB1	48.3	396.0	8.19	induced
YGL008C	PMA1	95.0	417.7	4.40	induced
YNL209W	SSB2	722.0	2015.3	2.79	induced
YDL246C	SOR2	143.7	358.7	2.50	induced

YDR155C	CPR1	994.7	1457.0	1.46	unchanged
YMR186W	HSC82	1344.3	1931.0	1.44	unchanged
YAL003W	EFB1	1555.7	1630.0	1.05	unchanged
YLR044C	PDC1	563.3	578.3	1.03	unchanged
YFL039C	ACT1	2874.7	2911.0	1.01	unchanged
YKL060C	FBA1	1213.7	1209.0	1.00	unchanged
YGR192C	TDH3	4115.0	4041.0	0.98	unchanged
YHR174W	ENO2	3446.3	3329.7	0.97	unchanged
YOR369C	RPS12	311.0	303.0	0.97	unchanged
YPL240C	HSP82	1785.7	1708.0	0.96	unchanged
YBR118W	TEF2	2034.0	1867.0	0.92	unchanged

```

induced: 4
unchanged: 11

```

Four genes are induced at least two-fold under stress; the rest barely move. Because the threshold is an option, the same tool answers a looser question without editing a line of code. Lowering it to 1.4 reclassifies the two borderline genes:

```
./expression_report.sh -e expression.tsv -a annotation.gtf -f 1.4
```

```

induced: 6
unchanged: 9

```

And a missing file is caught before any work begins:

```
./expression_report.sh -e nope.tsv -a annotation.gtf
```

```
error: 'nope.tsv' missing or empty
```

8.6 Making it reproducible

The script is one file, so the last steps from the previous chapter apply directly. Put it under Git together with a short `environment.yml` pinning the tools it uses, and a `.gitignore` that keeps generated reports out of the repository:

```

git init
git add expression_report.sh environment.yml .gitignore
git commit -m "Add expression_report.sh pipeline"

```

Anyone who clones the repository and runs `conda env create -f environment.yml` obtains the same tools at the same versions, and `./expression_report.sh` reproduces the same report from the same data. That is the full arc of this course in one place: raw files in, a defensible result out, by a procedure that is written down, version controlled, and repeatable.

8.7 Common mistakes

1. **Stages that disagree on the separator.** One `awk` emits tab-separated output, the next assumes spaces, and fields silently merge. Set `FS/OFS` consistently (`-F'\t', BEGIN{OFS="\t"}`) across the whole pipeline.
2. **Joining unsorted inputs.** Both branches must **sort** on the gene ID before the `join`, or rows go missing without warning.
3. **No intermediate inspection.** When a pipeline gives a surprising result, look at each stage's output in turn (write it to a file, or pipe the first stages to `head`) rather than guessing. Build a pipeline one stage at a time.
4. **Hard-coded paths and thresholds.** A tool with the file names and the cutoff buried in its body only works on one dataset. Take them as options, as the script above does.
5. **Leaving scratch files in the working directory.** Use `mktemp -d` and a `trap ... EXIT` so intermediate files live in a temporary directory that is removed automatically, keeping the working directory clean.
6. **Skipping validation.** A pipeline that assumes its inputs exist and are well-formed fails in confusing ways deep inside. Check the inputs up front and fail with a clear message.

8.8 Materials

All resources below are open-access.

- **The Unix shell** (Software Carpentry), a lesson on building pipelines and small shell programs for data, <https://swcarpentry.github.io/shell-novice/>
- **GNU Awk user's guide**, <https://www.gnu.org/software/gawk/manual/>
- **GNU Bash manual**, <https://www.gnu.org/software/bash/manual/>
- **Data Science at the Command Line** (Jeroen Janssens), free online, building analyses by composing command-line tools, <https://jeroenjanssens.com/dsatcl/>
- **The Turing Way**, reproducible research practices, <https://the-turing-way.netlify.app/>

8.9 Exercises

These exercises ask you to build a complete pipeline of your own, using everything from the course. Work from the data package; develop each stage at the prompt first, then assemble the script. Exercises 8.1 to 8.3 are the core of the chapter and are enough to show that you can build and run a pipeline. Exercises 8.4 to 8.6 are marked *going further*: they are optional, more open, and more ambitious, take them on if you would like a greater challenge.

Exercise 8.1. Run `expression_report.sh` on the course data and confirm the report above. Then run it with `-f 3` and with `-f 1.2`, and describe how the number of induced genes changes and why.

Exercise 8.2. Add a stage to the pipeline that, after classification, writes the induced genes to one file and the unchanged genes to another (hint: a final `awk` with output redirected per class, or a `grep` on the class column). Keep the printed summary as well.

Exercise 8.3. Write a second pipeline, `gc_report.sh`, that reads `transcripts.fa` and reports, for each sequence, its length and GC percentage, then prints the ten sequences with the highest GC content. Give it a `getopts` interface (`-i FILE`, `-n N` for how many to show) and run it under `set -euo pipefail`.

Exercise 8.4 (going further, optional). Combine annotation and sequence: using `annotation.gtf`, count how many exons each gene has, and report the five genes with the most exons together with their names. Build it as independent stages joined on the gene ID, as in this chapter.

Exercise 8.5 (going further, optional). Take any pipeline from the exercises above and make it reproducible: create a Git repository containing the script, a pinned `environment.yml`, a `.gitignore` that excludes generated output, and a short `README.md` documenting how to run it. Make at least two commits with meaningful messages and show the history with `git log --oneline`.

Exercise 8.6 (going further, open-ended, optional). Choose a question the course data can answer that is not covered above (for example, which chromosome has the highest gene density, or how transcript length relates to GC content), and build a complete, robust, documented tool that answers it. This is the deliverable that demonstrates the whole course: small tools, composed into a pipeline, wrapped as a reproducible script.